

C++ ile Dear ImGui

Immediate-Mode GUI ile Araç ve Arayüz Geliştirme Rehberi

Mimariden Widget'lara, Stilden Docking'e, Bol Projeli Kapsamlı Başvuru

Bu belge önce arayüz programlamanın **iki paradigmasını** (immediate-mode ve retained-mode), sonra Dear ImGui'nin **nasıl çalıştığını** (kare döngüsü, ID sistemi, DrawList), ardından **tüm önemli widget'ları** ve en sonda **sıfırdan yazılmış çok sayıda tam projeyi** adım adım anlatır. Tüm kod örnekleri İngilizce isimlendirmeyeyle, gerçek üretim stiline uygun yazılmıştır ve olduğu gibi derlenebilir.

Kapsam — Kısım 1 (Temeller): ImGui nedir • immediate vs retained • mimari & kare döngüsü • kurulum & CMake • GLFW+OpenGL3 backend • ilk pencere • ana döngü anatomisi • IO & DeltaTime.

Kapsam — Kısım 2 (Widget'lar): pencereler & bayraklar • buton/checkbox/radio • slider & drag • InputText • combo & listbox • renk seçici • ağaç & seçilebilir • tablolar • menü & tab bar • plot • ID sistemi & layout.

Kapsam — Kısım 3 (İleri): stil & tema • font & Türkçe karakter • docking & viewport • DrawList ile özel çizim • InputText callback • klavye/fare • ListClipper ile performans • yerleşik demo.

Kapsam — Kısım 4 (Projeler): yeniden kullanılabilir App iskeleti • Yapılacaklar (To-Do) uygulaması • Log konsolu • Canlı sistem monitörü • Renk paleti tasarımcısı • Hex/bellek görüntüleyici • Basit çizim tuvali • Oyun hata ayıklama HUD'u.

Hazırlanma Tarihi: 19 Temmuz 2026

C++17 • Dear ImGui 1.90+ • GLFW 3.x • OpenGL 3.3 • CMake

Contents

1	Dear ImGui Nedir?	4
1.1	Neye Yarar, Neye Yaramaz?	4
1.2	Kısa Tarihçe ve Ekosistem	4
2	Immediate Mode ve Retained Mode	5
2.1	Retained Mode: Nesneler Kalıcıdır	5
2.2	Immediate Mode: Her Karede Yeniden Söylersiniz	5
2.3	İki Yaklaşımın Karşılaştırması	5
3	ImGui'nin Mimarisi ve Kare Döngüsü	6
3.1	Üç Katman	6
3.2	Bir Karenin Anatomisi	6
4	Kurulum ve Proje Yapısı	7
4.1	Gerekli Dosyalar	7
4.2	Önerilen Klasör Düzeni	8
4.3	CMakeLists.txt — FetchContent ile Otomatik İndirme	8
5	İlk Program: Boş Bir ImGui Penceresi	9
5.1	Tam Kaynak Kod	9
5.2	Satır Satır Ne Oldu?	10
5.3	Ana Döngüyü Yeniden Kullanılabilir Yapmak	11
6	Pencereler ve Pencere Bayrakları	12
6.1	Temel Pencere Kullanımı	12
6.2	Sık Kullanılan Pencere Bayrakları	12
6.3	Konum ve Boyut Ayarlama	13
7	Temel Girdi Widget'ları	13
7.1	Metin ve Etiketler	13
7.2	Butonlar	13
7.3	Checkbox ve Radio	14
7.4	Slider ve Drag	14
7.5	Metin Girişi (InputText)	14
7.6	Combo (Açılır Liste) ve ListBox	15
7.7	Renk Seçiciler	15
8	Düzen (Layout) ve Gruplama	16
8.1	Yatay Düzen ve Boşluk	16
8.2	Gruplar ve Dikey Hizalama	16
8.3	Sütunlar: Table API	16
8.4	Genişlik Kontrolü ve Öge Genişliği	17
9	ID Sistemi: ImGui'nin En Kritik Kavramı	17
9.1	Problem: Aynı Etiket = Aynı ID	17
9.2	Çözüm 1: ## ile Gizli ID	17
9.3	Çözüm 2: PushID/PopID	17

10 Kapsayıcı Widget'lar	18
10.1 Ağaç Düğümleri (TreeNode)	18
10.2 Seçilebilir Öğeler (Selectable)	18
10.3 CollapsingHeader	19
10.4 Menüler ve Menü Çubuğu	19
10.5 Sekmeler (Tab Bar)	19
10.6 Açılır Pencereleer ve Modal'lar (Popup)	20
10.7 İpucu Balonları (Tooltip)	20
11 Tablolar: Güçlü Veri Gösterimi	20
11.1 Temel Tablo	20
11.2 Sıralanabilir ve Dondurulmuş Başlıklı Tablo	21
12 Grafikler ve Görselleştirme	21
12.1 Çizgi ve Histogram	21
12.2 İlerleme Çubuğu	22
13 Stil ve Tema	23
13.1 Hazır Temalar ve Genel Ayarlar	23
13.2 Renkleri Elle Ayarlamak	23
13.3 Yerel (Geçici) Stil Değişikliği: Push/Pop	23
14 Fontlar ve Türkçe Karakter Desteği	24
14.1 Türkçe Fontu Yükleme	24
14.2 Birden Fazla Font ve Simge Fontları	24
15 Docking ve Çoklu Viewport	25
15.1 Docking'i Etkinleştirme	25
15.2 Ana DockSpace Oluşturma	25
16 DrawList ile Özel Çizim	26
16.1 DrawList'e Erişim	26
16.2 Temel Şekiller	26
16.3 Etkileşimli Tuval Deseni	26
17 InputText Geri Çağırımları (Callback)	27
17.1 Callback Bayrakları ve İskelet	27
18 Klavye, Fare ve Öğe Durumu Sorgulama	28
18.1 Öğe Durumu Fonksiyonları	28
18.2 Doğrudan Klavye ve Fare	28
19 Performans: ListClipper ve İpuçları	29
19.1 ListClipper Kullanımı	29
19.2 Genel Performans İpuçları	29
19.3 Yerleşik Demo ve Yardımcı Pencereleer	29
20 Proje 0: Yeniden Kullanılabilir Uygulama İskeleti	30
20.1 app.h — İskelet	30
20.2 Kullanım Kalıbı	31
21 Proje 1: Yapılacaklar (To-Do) Uygulaması	31
21.1 Tam Kaynak	31

22 Proje 2: Renkli Log Konsolu	33
22.1 Tam Kaynak	33
23 Proje 3: Canlı Sistem Monitörü	35
23.1 Kayan Tampon (Scrolling Buffer)	35
24 Proje 4: Renk Paleti Tasarımcısı	36
24.1 Tam Kaynak	36
25 Proje 5: Hex / Bellek Görüntüleyici	38
25.1 Tam Kaynak	38
26 Proje 6: Basit Çizim Tuvali	39
26.1 Tam Kaynak	39
27 Proje 7: Oyun Hata Ayıklama HUD'u	41
27.1 Tam Kaynak	41
28 Hızlı Başvuru (Cheat Sheet)	44
28.1 Sık Kullanılan Çağrılar	44
28.2 Eşleşmesi Zorunlu Çiftler	44
29 Sık Yapılan Hatalar ve Çözümleri	45
30 Sonraki Adımlar	45

KISIM 1

Dear ImGui'nin Temelleri

1. Dear ImGui Nedir?

Dear ImGui (kısaca ImGui, “Immediate Mode Graphical User Interface”), C++ ile yazılmış, bağımlılığı olmayan (dependency-free) bir arayüz kütüphanesidir. Omar Cornut tarafından geliştirilir. Amacı; oyun motorları, grafik uygulamaları, hata ayıklama araçları ve içerik editörleri için **hızlıca üretilen, minimum durum (state) tutan** arayüzler yaratmaktır. Bugün Unreal Engine'den Blender eklentilerine, NVIDIA araçlarından bağımsız oyun stüdyolarına kadar sektörde çok yaygın kullanılır.

ImGui bir *işletim sistemi* arayüzü (Qt, GTK gibi) değildir. Kendine ait bir pencereyi işletim sisteminden açmaz; bunun yerine sizin zaten sahip olduğunuz bir OpenGL/DirectX/Vulkan bağlamına (context) **üçgenler çizerek** arayüz üretir. Bu yüzden ona genelde bir oyun ya da render döngüsünün *içine* yerleştiririz.

1.1 Neye Yarar, Neye Yaramaz?

İdeal olduğu yerler

Oyun içi hata ayıklama panelleri, editörler
Grafik/render araçları, shader denemeleri
Bilimsel görselleştirme, veri denetleyicileri
Hızlı prototipleme, “bir slider koyayım” anları

Uygun olmadığı yerler

Son kullanıcıya sunulan cilalı masaüstü uygulaması (bazen yine de kullanılır)
Mobil dokunmatik odaklı tüketici uygulamaları
Karmaşık, tema-ağırlıklı web benzeri UI'lar
Erişilebilirlik (a11y) zorunlu kurumsal yazılım

Bilgi

ImGui'nin sloganı “bloat-free”dir. Bir değeri ekrana bağlamak için gözlemci (observer), sinyal-slot, XML layout dosyası veya kod üretici *yoktur*. Bir if bloğu kadar basittir: `if (ImGui::Button("Kaydet")) save();`

1.2 Kısa Tarihçe ve Ekosistem

ImGui 2014'te açık kaynak oldu ve MIT lisanslıdır. Etrafında zengin bir ekosistem büyüdü:

Bileşen	Açıklama
Çekirdek (imgui.h)	Tüm widget'lar, düzen, girdi işleme. Platformdan bağımsız.
Backend'ler	imgui_impl_glfw, imgui_impl_opengl3, _dx11, _vulkan, _sdl2 vb. Girdiyi ve çizimi platforma bağlar.
ImPlot	Yüksek performanslı bilimsel grafik/çizim eklentisi.
ImGuizmo	3B sahnede taşıma/döndürme/ölçekleme gizmaları.
imgui-node-editor	Düğüm (node) tabanlı grafik editörleri.

2. Immediate Mode ve Retained Mode

ImGui'yi gerçekten anlamamanın anahtarı, onun “immediate mode” (anlık kip) felsefesini kavramaktır. Qt, GTK, HTML/DOM, Android View gibi klasik arayüz sistemleri **retained mode** (kalıcı kip) çalışır. İki arasındaki fark, kütüphaneyle kurduğunuz ilişkinin tamamını belirler.

2.1 Retained Mode: Nesneler Kalıcıdır

Retained mode'da bir düğme oluşturursunuz ve o düğme nesnesi *bellekte yaşamaya devam eder*. Ona bir metin verir, bir tıklama olayı bağlar (callback) ve sonra unutursunuz. Arayüz ağacı (widget tree) bir veri yapısı olarak saklanır; siz onu değiştirdiğinizde kütüphane ekranı günceller.

```

1 // Retained mode (kavramsal, Qt benzeri sozde-kod)
2 QPushButton* button = new QPushButton("Kaydet"); // nesne kalıcı oluşturulur
3 button->setEnabled(false); // durumu kutuphane tutar
4 connect(button, &QPushButton::clicked, this, &App::onSave); // olay bağlanır
5 layout->addWidget(button); // ağaca eklenir
6 // ... buradan sonra 'button' bellekte yasar, olaylari bekler

```

2.2 Immediate Mode: Her Karede Yeniden Söylersiniz

ImGui'de düğme diye bir nesne *yoktur*. Her karede (frame), “şu anda ekranda bir Kaydet düğmesi olsun” dersiniz. Fonksiyon o karede düğmeyi çizer ve o karede tıklamp tıklanmadığını bool olarak *hemen* döndürür. Bir sonraki kare, aynı satırı tekrar çağırırsınız.

```

1 // Immediate mode (gerçek ImGui)
2 if (ImGui::Button("Kaydet")) { // hem çizer hem "bu kare tiklandi mi?" doner
3     save(); // tiklandiyse hemen burada is yapariz
4 }
5 // Sonraki kare bu satir yine calisir. Dugme icin saklanan bir nesne YOK.

```

Zihniyet değişimi

Immediate mode'da arayüz, uygulama durumunuzun (state) her kare yeniden çizilen bir *görüntüsüdür*. “Düğmeyi devre dışı bırak” gibi bir komut yollamaz, sadece “bu karede düğme devre dışı görünsün” dersiniz: `ImGui::BeginDisabled(isBusy); ... ImGui::EndDisabled();`. Senkronizasyon problemi ortadan kalkar: gösterdiğiniz şey her zaman verinizin o anki hâlidir.

2.3 İki Yaklaşımın Karşılaştırması

Konu	Retained (Qt/HTML)	Immediate (ImGui)
Durum sahibi	Kütüphane widget ağacını tutar	Siz tutarsınız, UI sadece yansıtır
Olay modeli	Callback / sinyal-slot	Fonksiyon dönüş değeri (bool)
Bellek	Widget başına kalıcı nesne	Kare başına geçici, minimum
Senkronizasyon	UI ile veri ayrışabilir (bug kaynağı)	Ayrışamaz; her kare yeniden okunur
Performans profili	Az widget'ta ucuz	Her kare tüm UI çalışır (çoğu araç için sorun değil)
En iyi olduğu yer	Statik, karmaşık son-kullanıcı UI	Dinamik, sık değişen araç/editör UI

Dikkat

Immediate mode “her kare tüm arayüzü yeniden inşa ediyorum” demektir. Bu kulağa pahalı gelir ama modern donanımda binlerce widget bile mikrosaniyeler sürer, çünkü ImGui yalnızca üçgen listesi üretir. Yine de *ağır hesapları* (dosya okuma, ağ) UI kodunun içine koymayın; onları ayrı yapıp sonucu gösterin.

3. ImGui'nin Mimarisi ve Kare Döngüsü

ImGui üç katmandan oluşur ve bu katmanları ayırt etmek kurulumu anlamayı kolaylaştırır. Sizin uygulamanız bu üç katmanı her karede belli bir sırayla sürer.

3.1 Üç Katman

Katman	Sorumluluk
1. Çekirdek ImGui	Widget mantığı, düzen, girdi durumu. <code>ImGui::Button</code> , <code>ImGui::Begin</code> burada. Çıktısı: bir çizim komut listesi (<code>DrawData</code>).
2. Platform backend	Klavye, fare, pencere boyutu, zamanı ImGui'ye <i>besler</i> . Örn. <code>imgui_impl_glfw</code> .
3. Renderer backend	ImGui'nin ürettiği üçgenleri GPU'ya çizer. Örn. <code>imgui_impl_opengl3</code> .

ImGui hiçbir zaman doğrudan ekrana çizmez. Bir karenin sonunda, `ImGui::Render()` çağrısı bir `ImDrawData` nesnesi hazırlar: bu, “şu köşe tamponunu şu dokuyla, şu kırpma dikdörtgeninde çiz” diyen komutların listesidir. Renderer backend bu listeyi alıp OpenGL çağrılarına çevirir.

3.2 Bir Karenin Anatomisi

Her kare tam olarak şu sırayı izler. Bu sırayı bozmak (örneğin `NewFrame` çağırmadan widget kullanmak) çökmeye yol açar.

Adım	Yapılan iş
1	Platform olaylarını işle (glfwPollEvents).
2	Backend'lerin NewFrame'i: yeni girdiyi ImGui'ye aktar.
3	ImGui::NewFrame(): yeni kareyi başlat.
4	Sizin UI kodunuz: Begin/End, tüm widget çağrıları.
5	ImGui::Render(): DrawData'yı üret.
6	Ekranı temizle (glClear).
7	Renderer backend: DrawData'yı çiz.
8	Tamponları takasla (glfwSwapBuffers).

Bilgi

4. adım, yani sizin UI kodunuz, tamamen NewFrame ile Render arasında olmak zorundadır. Bu ikisinin arasına yazdığımız her ImGui:: çağrısı o kareye ait UI'yı tanımlar. Dışına yazarsanız ImGui "hangi karedeyim?" diyemez ve assert ile durur.

4. Kurulum ve Proje Yapısı

ImGui'nin bir "kur" komutu yoktur; kaynak dosyaları doğrudan projenize eklersiniz. Bu, kütüphanenin felsefesinin bir parçasıdır: derlenmiş bir kütüphane (.lib/.so) yerine *birkaç .cpp dosyasını* kendi projenizle beraber derlersiniz. Bu bölümde GLFW + OpenGL3 backend'i ile modern bir CMake projesi kuracağız.

4.1 Gerekli Dosyalar

Depodan (<https://github.com/ocornut/imgui>) şu dosyalara ihtiyacımız var:

Dosya	Rol
imgui.cpp / imgui.h	Çekirdek kütüphane
imgui_draw.cpp	Çizim komutları ve DrawList
imgui_tables.cpp	Tablo widget'ları
imgui_widgets.cpp	Standart widget'lar
imgui_demo.cpp	Demo penceresi (öğrenmek için altın değerinde)
backends/imgui_impl_glfw.cpp	GLFW platform backend
backends/imgui_impl_opengl3.cpp	OpenGL3 renderer backend

İpucu

imgui_demo.cpp içindeki ImGui::ShowDemoWindow(), neredeyse her widget'ın canlı bir örneğini gösterir. Öğrenirken demo penceresini açın, beğendiğiniz widget'ı bulun, sonra imgui_demo.cpp içinde o metni aratıp kodunu kopyalayın. Bu, ImGui öğrenmenin en hızlı yoludur.

4.2 Önerilen Klasör Düzeni

```

1  imgui_proje/
2    CMakeLists.txt
3    src/
4      main.cpp
5    external/
6      imgui/          <- ocornut/imgui deposu (git submodule ideal)
7      glfw/           <- glfw/glfw deposu (veya sistemden find_package)

```

4.3 CMakeLists.txt — FetchContent ile Otomatik İndirme

En pratik yöntem, CMake'in FetchContent modülüyle ImGui ve GLFW'yi otomatik indirmektir. Böylece kimse elle dosya kopyalamaz.

```

1  cmake_minimum_required(VERSION 3.16)
2  project(imgui_proje LANGUAGES CXX)
3
4  set(CMAKE_CXX_STANDARD 17)
5  set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7  include(FetchContent)
8
9  # --- GLFW ---
10 FetchContent_Declare(glfw
11   GIT_REPOSITORY https://github.com/glfw/glfw.git
12   GIT_TAG         3.4)
13 set(GLFW_BUILD_DOCS OFF CACHE BOOL "" FORCE)
14 set(GLFW_BUILD_TESTS OFF CACHE BOOL "" FORCE)
15 set(GLFW_BUILD_EXAMPLES OFF CACHE BOOL "" FORCE)
16 FetchContent_MakeAvailable(glfw)
17
18 # --- Dear ImGui ---
19 FetchContent_Declare(imgui
20   GIT_REPOSITORY https://github.com/ocornut/imgui.git
21   GIT_TAG         v1.91.5)
22 FetchContent_MakeAvailable(imgui)
23
24 # ImGui bir CMake hedefi sunmaz; kaynaklari elle derleriz.
25 add_library(imgui STATIC
26   ${imgui_SOURCE_DIR}/imgui.cpp
27   ${imgui_SOURCE_DIR}/imgui_draw.cpp
28   ${imgui_SOURCE_DIR}/imgui_tables.cpp
29   ${imgui_SOURCE_DIR}/imgui_widgets.cpp
30   ${imgui_SOURCE_DIR}/imgui_demo.cpp
31   ${imgui_SOURCE_DIR}/backends/imgui_impl_glfw.cpp
32   ${imgui_SOURCE_DIR}/backends/imgui_impl_opengl3.cpp)
33
34 target_include_directories(imgui PUBLIC
35   ${imgui_SOURCE_DIR}
36   ${imgui_SOURCE_DIR}/backends)
37
38 target_link_libraries(imgui PUBLIC glfw)
39
40 # --- Uygulamamiz ---
41 add_executable(app src/main.cpp)
42 target_link_libraries(app PRIVATE imgui)
43

```

```

44 # OpenGL'i bagla (Linux/macOS/Windows farkli)
45 find_package(OpenGL REQUIRED)
46 target_link_libraries(app PRIVATE OpenGL::GL)

```

```

1 # Derleme
2 cmake -B build -DCMAKE_BUILD_TYPE=Release
3 cmake --build build -j
4 ./build/app

```

Bilgi

ImGui, OpenGL fonksiyon işaretçilerini yüklemek için varsayılan olarak kendi hafif yükleyicisini (imgui_impl_opengl3_loader.h) kullanır; bu yüzden GLAD/GLEW eklemenize gerek yoktur. Kendi çiziminizi (sahne) OpenGL ile yaparsanız yine de bir yükleyici istersiniz, ama sadece ImGui için gerekmez.

5. İlk Program: Boş Bir ImGui Penceresi

Şimdi en küçük tam programı yazalım. Bu program bir GLFW penceresi açar, ImGui'yi başlatır ve içinde tek bir metin gösteren bir ImGui penceresi çizer. Bu iskelet, rehberdeki her örneğin temelidir.

5.1 Tam Kaynak Kod

```

1 // main.cpp -- en küçük calisan ImGui uygulaması
2 #include "imgui.h"
3 #include "imgui_impl_glfw.h"
4 #include "imgui_impl_opengl3.h"
5 #include <GLFW/glfw3.h>
6 #include <cstdio>
7
8 static void glfw_error_callback(int error, const char* description) {
9     std::fprintf(stderr, "GLFW Hata %d: %s\n", error, description);
10 }
11
12 int main() {
13     // 1) GLFW'yi baslat
14     glfwSetErrorCallback(glfw_error_callback);
15     if (!glfwInit()) return 1;
16
17     // OpenGL 3.3 Core baglami iste
18     const char* glsl_version = "#version 330";
19     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
20     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
21     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
22
23     GLFWwindow* window = glfwCreateWindow(1280, 720,
24                                         "ImGui İlk Pencere", nullptr, nullptr);
25     if (!window) { glfwTerminate(); return 1; }
26     glfwMakeContextCurrent(window);
27     glfwSwapInterval(1); // VSync acik
28
29     // 2) ImGui baglamini olustur
30     ImGui::CheckVersion();
31     ImGui::CreateContext();

```

```

32     ImGuiIO& io = ImGui::GetIO();
33     io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard; // klavye ile gezinme
34
35     ImGui::StyleColorsDark(); // koyu tema
36
37     // 3) Backend'leri baslat
38     ImGui_ImplGlfw_InitForOpenGL(window, true);
39     ImGui_ImplOpenGL3_Init(glsl_version);
40
41     // 4) Ana dongu
42     while (!glfwWindowShouldClose(window)) {
43         glfwPollEvents();
44
45         // Yeni kareyi baslat
46         ImGui_ImplOpenGL3_NewFrame();
47         ImGui_ImplGlfw_NewFrame();
48         ImGui::NewFrame();
49
50         // ---- BURADAN itibaren bizim UI kodumuz ----
51         ImGui::Begin("Merhaba");
52         ImGui::Text("Merhaba, Dear ImGui!");
53         ImGui::Text("Uygulama %.1f FPS calisiyor", io.Framerate);
54         ImGui::End();
55         // ---- UI kodu bitti ----
56
57         // Kareyi render et
58         ImGui::Render();
59         int display_w, display_h;
60         glfwGetFramebufferSize(window, &display_w, &display_h);
61         glViewport(0, 0, display_w, display_h);
62         glClearColor(0.10f, 0.11f, 0.13f, 1.0f);
63         glClear(GL_COLOR_BUFFER_BIT);
64         ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
65
66         glfwSwapBuffers(window);
67     }
68
69     // 5) Temizlik
70     ImGui_ImplOpenGL3_Shutdown();
71     ImGui_ImplGlfw_Shutdown();
72     ImGui::DestroyContext();
73     glfwDestroyWindow(window);
74     glfwTerminate();
75     return 0;
76 }

```

5.2 Satır Satır Ne Oldu?

Bölüm	Açıklama
CreateContext	ImGui'nin tüm iç durumunu tutan tekil (singleton) bağlamı yaratır. Her şeyden önce çağrılmalı.
GetIO	Girdi/çıkı yapıları: fare, klavye, ekran boyutu, DeltaTime, framerate buradadır.
Init...ForOpenGL	GLFW backend'i pencereye kancalar; true argümanı ImGui'nin girdi callback'lerini otomatik kurmasını sağlar.
Begin/End	Bir ImGui penceresi açar ve kapatır. İçindeki widget'lar bu pencereye çizilir.
Render + RenderDrawData	UI'yı üçgen listesine çevirir ve OpenGL ile çizer.

Dikkat

ImGui::Begin her zaman bir ImGui::End ile eşleşmelidir — Begin false dönse bile! Bu, çoğu BeginXXX/EndXXX çiftinden farklıdır (birçoğunda Begin false dönerse End çağrılmaz). Pencere için kural: Begin ne dönerse dönsün End çağır. Güvenli kalıp: `if (ImGui::Begin("X")) { ... }` ImGui::End();

5.3 Ana Döngüyü Yeniden Kullanılabilir Yapmak

Her projede yukarıdaki iskeleti tekrar yazmak istemeyiz. Kısım 4'te bu döngüyü bir App sınıfına saracağız; böylece sadece “bu karede ne çizilsin” sorusuna odaklanabileceğiz. Şimdilik yukarıdaki main.cpp tüm örnekler için yeterli — sadece *UI kodu* bölümünü değiştireceğiz.

KISIM 2

Widget'lar ve Düzen

6. Pencereler ve Pencere Bayrakları

Her ImGui widget'ı bir pencerenin içinde yaşar. `ImGui::Begin("Ad")` bir pencere açar; aynı ada sahip `Begin` çağrılar *aynı* pencereye yazar. Pencerenin başlığı aynı zamanda onun kimliğidir (bkz. ID sistemi).

6.1 Temel Pencere Kullanımı

```
1 bool show_panel = true;
2
3 // 2. argümanla bir kapatma düğmesi (X) ekleyebiliriz:
4 if (ImGui::Begin("Ayarlar", &show_panel)) {
5     ImGui::Text("Panel icerigi burada.");
6 }
7 ImGui::End();
8 // show_panel, kullanıcı X'e basınca false olur -> bir daha çizmezsiniz.
```

6.2 Sık Kullanılan Pencere Bayrakları

`Begin`'in üçüncü argümanı `ImGuiWindowFlags` bit maskesidir.

Bayrak	Etki
<code>NoTitleBar</code>	Başlık çubuğunu gizler
<code>NoResize</code>	Yeniden boyutlandırmayı kapatır
<code>NoMove</code>	Pencere taşınamaz
<code>NoCollapse</code>	Katlama (çift tık) okunu kaldırır
<code>AlwaysAutoResize</code>	Pencere içeriğe göre boyutlanır
<code>NoBackground</code>	Arka planı şeffaf yapar (HUD için ideal)
<code>MenuBar</code>	Pencereye bir menü çubuğu ekler
<code>NoScrollbar</code>	Kaydırma çubuğunu gizler
<code>NoSavedSettings</code>	Konum/boyutu <code>imgui.ini</code> 'ye kaydetmez

```

1 ImGuiWindowFlags flags = ImGuiWindowFlags_NoResize
2                       | ImGuiWindowFlags_NoCollapse
3                       | ImGuiWindowFlags_AlwaysAutoResize;
4 ImGui::Begin("Sabit Panel", nullptr, flags);
5 ImGui::Text("Bu pencere boyutlandırılmaz ve katlanamaz.");
6 ImGui::End();

```

6.3 Konum ve Boyut Ayarlama

Pencere konumu/boyutu `imgui.ini`'de saklanır ve kullanıcı taşıyabilir. Bir başlangıç değeri vermek için `ImGuiCond` kullanırız: `_Once` sadece ilk açılışta, `_Always` her karede uygular.

```

1 ImGui::SetNextWindowPos(ImVec2(20, 20), ImGuiCond_FirstUseEver);
2 ImGui::SetNextWindowSize(ImVec2(400, 300), ImGuiCond_FirstUseEver);
3 ImGui::Begin("Konumlu Pencere");
4 ImGui::End();

```

Dikkat

Konumu `ImGuiCond_Always` ile her karede zorlarsanız kullanıcı pencereyi taşıyamaz — taşısa da bir sonraki karede geri sıçrar. Başlangıç konumu için *daima* `_FirstUseEver` ya da `_Once` kullanın.

7. Temel Girdi Widget'ları

Bu bölümde en çok kullanılan girdi widget'larını tek tek görelim. Ortak desen: widget bir `bool` döner (etkileşim oldu mu?) ve bir referans aldığı değişkeni *yerinde* günceller.

7.1 Metin ve Etiketler

```

1 ImGui::Text("Duz metin");
2 ImGui::TextColored(ImVec4(1,0.6f,0,1), "Turuncu vurgulu metin");
3 ImGui::TextDisabled("Soluk (devre disi gorunumlu) metin");
4 ImGui::TextWrapped("Cok uzun bir metin otomatik satir kaydirilir...");
5 ImGui::BulletText("Madde imli satir");
6 ImGui::LabelText("FPS", "%.1f", 59.9f); // sag: etiket, sol: deger
7 ImGui::SeparatorText("Bolum Basligi"); // ortasinda yazi olan cizgi
8
9 // printf tarzi bicimlendirme:
10 int score = 42;
11 ImGui::Text("Skor: %d", score);

```

7.2 Butonlar

```

1 if (ImGui::Button("Kaydet")) {
2     // tiklandi
3 }
4 ImGui::SameLine(); // sonraki widget ayni satirda
5 if (ImGui::Button("Iptal")) { }
6
7 ImGui::SmallButton("Kucuk"); // satir ici kucuk dugme
8 ImGui::Button("Genis", ImVec2(-FLT_MIN, 0)); // -FLT_MIN: mevcut genisligi doldur
9 ImGui::ArrowButton("ok", ImGuiDir_Right); // yon oklu dugme
10

```

```

11 // Basili tutuldukca tekrar tetiklenen dugme:
12 ImGui::PushButtonRepeat(true);
13 if (ImGui::Button("++")) counter++;
14 ImGui::PopButtonRepeat();

```

7.3 Checkbox ve Radio

```

1 static bool vsync = true;
2 ImGui::Checkbox("VSync acik", &vsync);
3
4 static int mode = 0;
5 ImGui::RadioButton("Dusuk", &mode, 0); ImGui::SameLine();
6 ImGui::RadioButton("Orta", &mode, 1); ImGui::SameLine();
7 ImGui::RadioButton("Yuksek", &mode, 2);
8
9 // Bit bayraklari icin CheckboxFlags:
10 static int flags = 0;
11 ImGui::CheckboxFlags("Bayrak A", &flags, 1 << 0);
12 ImGui::CheckboxFlags("Bayrak B", &flags, 1 << 1);

```

static neden?

Örneklerde static kullanmamız, değerin kareler arası yaşaması içindir — immediate mode'da yerel değişken her kare sıfırlanır. **Gerçek uygulamada** bu değerleri static değil, kendi uygulama/durum sınıfımızın üyeleri yapın. static, sadece tek örnekli hızlı denemeler için uygundur.

7.4 Slider ve Drag

Slider bir aralıkta değer seçtirir; Drag ise sınırsız (isteğe bağlı sınırlı) değeri fare sürükleyerek değiştirir.

```

1 static float volume = 0.5f;
2 ImGui::SliderFloat("Ses", &volume, 0.0f, 1.0f);
3
4 static int level = 3;
5 ImGui::SliderInt("Seviye", &level, 1, 10);
6
7 static float pos[3] = {0,0,0};
8 ImGui::SliderFloat3("Konum", pos, -10.0f, 10.0f); // 3 bileşenli
9
10 static float speed = 1.0f;
11 ImGui::DragFloat("Hiz", &speed, 0.01f /*adım*/, 0.0f, 100.0f);
12
13 static int range[2] = {10, 90};
14 ImGui::DragIntRange2("Aralik", &range[0], &range[1], 1, 0, 100);
15
16 // Bicim ve logaritmik olcek:
17 ImGui::SliderFloat("Frekans", &speed, 1.0f, 20000.0f,
18                 "%.0f Hz", ImGuiSliderFlags_Logarithmic);

```

7.5 Metin Girişi (InputText)

InputText, C-tarzı bir char tamponuna yazar. std::string ile kullanmak için imgui_stdlib.h (misc/cpp klasöründe) eklenir.

```

1 static char name[128] = "";

```

```

2 ImGui::InputText("Isim", name, IM_ARRAYSIZE(name));
3
4 static char password[64] = "";
5 ImGui::InputText("Sifre", password, IM_ARRAYSIZE(password),
6                 ImGuiInputTextFlags_Password);
7
8 static char notes[1024] = "";
9 ImGui::InputTextMultiline("Notlar", notes, IM_ARRAYSIZE(notes),
10                          ImVec2(-FLT_MIN, 100));
11
12 // Sayisal giris:
13 static int age = 25;
14 ImGui::InputInt("Yas", &age);
15 static float pi = 3.14f;
16 ImGui::InputFloat("Pi", &pi, 0.01f, 1.0f, "%.3f");

```

İpucu

std::string kullanmak için: #include "misc/cpp/imgui_stdlib.h" ekleyin ve std::string s; ImGui::InputText("X", &s); yazın. Bu, tampon boyutu derdinden kurtarır. Kısım 3'te InputText callback'lerini de göreceğiz.

7.6 Combo (Açılır Liste) ve ListBox

```

1 // Basit yol: null ile ayrılmış tek string
2 const char* items = "Kirmizi\0Yesil\0Mavi\0\0";
3 static int current = 0;
4 ImGui::Combo("Renk", &current, items);
5
6 // Dizi ile:
7 const char* names[] = {"Dusuk", "Orta", "Yuksek", "Ultra"};
8 static int quality = 1;
9 ImGui::Combo("Kalite", &quality, names, IM_ARRAYSIZE(names));
10
11 // Tam kontrol için BeginCombo/EndCombo:
12 static int sel = 0;
13 if (ImGui::BeginCombo("Gelismis", names[sel])) {
14     for (int i = 0; i < IM_ARRAYSIZE(names); i++) {
15         bool is_selected = (sel == i);
16         if (ImGui::Selectable(names[i], is_selected)) sel = i;
17         if (is_selected) ImGui::SetItemDefaultFocus();
18     }
19     ImGui::EndCombo();
20 }
21
22 // ListBox: birden fazla ogeyi ayni anda gorunur kutuda gosterir
23 static int lb = 0;
24 ImGui::ListBox("Liste", &lb, names, IM_ARRAYSIZE(names), 4 /*gorunur satir*/);

```

7.7 Renk Seçiciler

```

1 static float color3[3] = {1.0f, 0.5f, 0.2f};
2 ImGui::ColorEdit3("Renk (RGB)", color3);
3
4 static float color4[4] = {1.0f, 0.5f, 0.2f, 1.0f};
5 ImGui::ColorEdit4("Renk (RGBA)", color4);

```



```

6
7 // Büyük tekerlekli secici:
8 ImGui::ColorPicker4("Secici", color4);
9
10 // Sadece renk kutusu (secici acmadan):
11 ImGui::ColorButton("onizleme", ImVec4(color4[0],color4[1],color4[2],color4[3]));

```

8. Düzen (Layout) ve Gruplama

ImGui varsayılan olarak widget'ları alt alta dizer. Yatay düzen, boşluk, girinti ve gruplama için özel komutlar vardır.

8.1 Yatay Düzen ve Boşluk

```

1 ImGui::Text("A"); ImGui::SameLine(); ImGui::Text("B"); // yan yana
2 ImGui::SameLine(0, 40); // aralarında 40 piksel boşluk
3 ImGui::Text("C (40px sonra)");
4
5 ImGui::Spacing(); // dikey boşluk
6 ImGui::Separator(); // yatay ayırıcı çizgi
7 ImGui::NewLine(); // boş satır
8
9 ImGui::Indent(); // girinti ekle
10 ImGui::Text("Girintili");
11 ImGui::Unindent(); // girintiyi geri al
12
13 // Bir sonraki widget'ın konumunu elle ayarla:
14 ImGui::SetCursorPosX(200);
15 ImGui::Text("x=200'de");

```

8.2 Gruplar ve Dikey Hizalama

BeginGroup/EndGroup bir grup widget'ı tek bir öğe gibi ele alır; böylece SameLine onları bütün olarak yan yana koyabilir.

```

1 ImGui::BeginGroup();
2 ImGui::Text("Sol sütun");
3 ImGui::Button("Dugme 1");
4 ImGui::Button("Dugme 2");
5 ImGui::EndGroup();
6
7 ImGui::SameLine();
8
9 ImGui::BeginGroup();
10 ImGui::Text("Sag sütun");
11 ImGui::Button("Dugme 3");
12 ImGui::EndGroup();

```

8.3 Sütunlar: Table API

Eski Columns() API'si yerine artık esnek **Table API**'si önerilir. Basit bir iki sütunlu düzen:

```

1 if (ImGui::BeginTable("layout", 2)) {
2     ImGui::TableNextColumn(); ImGui::Text("Etiket 1");
3     ImGui::TableNextColumn(); ImGui::Text("Deger 1");

```

```

4   ImGui::TableNextColumn(); ImGui::Text("Etiket 2");
5   ImGui::TableNextColumn(); ImGui::Text("Deger 2");
6   ImGui::EndTable();
7 }

```

8.4 Genişlik Kontrolü ve Öge Genişliği

```

1  ImGui::PushItemWidth(120);           // sonraki widget'lar 120 px genis
2  ImGui::SliderFloat("A", &a, 0, 1);
3  ImGui::SliderFloat("B", &b, 0, 1);
4  ImGui::PopItemWidth();
5
6  ImGui::SetNextItemWidth(-100);       // saga 100 px bosluk birakip doldur
7  ImGui::InputText("Ara", buf, 64);

```

Bilgi

Genişlik değerlerinde işaret önemlidir: pozitif değer sabit piksel genişliğidir; negatif değer “sağdan şu kadar boşluk bırak, gerisini doldur” demektir. `-FLT_MIN` ise “tüm kalan genişliği doldur” için yaygın bir kısayoldur.

9. ID Sistemi: ImGui'nin En Kritik Kavramı

ImGui her widget'ı bir **ID** ile takip eder (hangi öge aktif, hangisine tıklandı vb.). ID varsayılan olarak widget'ın *etiketinden* üretilir. Bu, yeni başlayanların en sık düştüğü tuzağın kaynağıdır.

9.1 Problem: Aynı Etiket = Aynı ID

```

1  // HATALI: iki dugme ayni "Sil" etiketine sahip -> ayni ID!
2  // ImGui ikisini tek dugme sanar; ikincisi calismayabilir.
3  for (int i = 0; i < 3; i++) {
4      if (ImGui::Button("Sil")) { erase(i); }    // <- ID cakismasi
5  }

```

9.2 Çözüm 1: ## ile Gizli ID

Etiketle ## sonrası kısım ekranda *görünmez* ama ID'ye katılır.

```

1  for (int i = 0; i < 3; i++) {
2      // Ekranda "Sil" görünür, ID benzersizdir:
3      char label[32];
4      snprintf(label, sizeof(label), "Sil###%d", i);
5      if (ImGui::Button(label)) { erase(i); }
6  }

```

9.3 Çözüm 2: PushID/PopID

Daha temiz yöntem: döngü gövdesini bir ID kapsamına almak.

```

1  for (int i = 0; i < 3; i++) {
2      ImGui::PushID(i);           // bu iterasyona ozgu ID kapsamı
3      if (ImGui::Button("Sil")) { erase(i); }    // etiket ayni ama ID benzersiz
4      ImGui::SameLine();

```

```

5   ImGui::Text("Oge %d", i);
6   ImGui::PopID();
7 }

```

ve ### farkı

"Baslik##gizli": görünen metin "Baslik", ID ise tüm dizeden üretilir. "###sabitID": görünen metin boşalır ve *sadece* ### sonrası ID belirler — bu, etiketi her karede değişen (örn. sayaç gösteren) bir pencerenin kimliğini sabit tutmak için kullanılır: `ImGui::Begin("Skor: 42###skorPenceresi");`

Dikkat

ID çakışması sessiz hatalara yol açar: yanlış widget aktifleşir, tıklamalar kaybolur, InputText odağı atlar. Bir liste/döngü içinde widget üretiyorsanız **daima** PushID/PopID kullanın. Bu, ImGui'de en sık karşılaşılan hata sınıfıdır.

10. Kapsayıcı Widget'lar

Bu widget'lar başka widget'ları içinde barındırır: ağaçlar, seçilebilir listeler, menüler, sekmeler ve açılır kutular.

10.1 Ağaç Düğümleri (TreeNode)

```

1  if (ImGui::TreeNode("Kok")) {
2      ImGui::Text("Kok altındaki icerik");
3      if (ImGui::TreeNode("Alt dal")) {
4          ImGui::BulletText("Yaprak 1");
5          ImGui::BulletText("Yaprak 2");
6          ImGui::TreePop();           // her acilan TreeNode icin bir TreePop
7      }
8      ImGui::TreePop();
9  }
10
11 // Bayraklarla: varsayılan açık, tıklamayla seçim
12 ImGuiTreeNodeFlags flags = ImGuiTreeNodeFlags_DefaultOpen
13                          | ImGuiTreeNodeFlags_OpenOnArrow;
14 if (ImGui::TreeNodeEx("Gelismis Dal", flags)) {
15     ImGui::Text("...");
16     ImGui::TreePop();
17 }

```

10.2 Seçilebilir Öğeler (Selectable)

```

1  static int selected = -1;
2  const char* files[] = {"main.cpp", "app.h", "utils.cpp", "readme.md"};
3  for (int i = 0; i < IM_ARRAYSIZE(files); i++) {
4      if (ImGui::Selectable(files[i], selected == i)) {
5          selected = i;           // tiklanınca sec
6      }
7      // Cift tik algıla:
8      if (ImGui::IsItemHovered() && ImGui::IsMouseClicked(0)) {
9          open_file(files[i]);

```

```

10 }
11 }

```

10.3 CollapsingHeader

Katlanabilir başlık; ayar panelleri için idealdir. TreePop gerektirmez.

```

1 if (ImGui::CollapsingHeader("Grafik Ayarlari")) {
2     ImGui::Checkbox("VSync", &vsync);
3     ImGui::SliderInt("MSAA", &msaa, 0, 8);
4 }
5 if (ImGui::CollapsingHeader("Ses Ayarlari",
6                             ImGuiTreeNodeFlags_DefaultOpen)) {
7     ImGui::SliderFloat("Ana Ses", &master, 0, 1);
8 }

```

10.4 Menüler ve Menü Çubuğu

Pencere menüsü için pencereye ImGuiWindowFlags_MenuBar, uygulama geneli için BeginMainMenuBar kullanılır.

```

1 // Uygulama ust menuyu (tum ekranin ustunde):
2 if (ImGui::BeginMainMenuBar()) {
3     if (ImGui::BeginMenu("Dosya")) {
4         if (ImGui::MenuItem("Yeni", "Ctrl+N")) { new_file(); }
5         if (ImGui::MenuItem("Ac", "Ctrl+O")) { open(); }
6         ImGui::Separator();
7         if (ImGui::MenuItem("Cikis", "Alt+F4")) { quit(); }
8         ImGui::EndMenu();
9     }
10    if (ImGui::BeginMenu("Duzen")) {
11        static bool wrap = true;
12        ImGui::MenuItem("Satir kaydir", nullptr, &wrap); // isaretli oge
13        ImGui::EndMenu();
14    }
15    ImGui::EndMainMenuBar();
16 }

```

10.5 Sekmeler (Tab Bar)

```

1 if (ImGui::BeginTabBar("Sekmeler")) {
2     if (ImGui::BeginTabItem("Genel")) {
3         ImGui::Text("Genel ayarlar");
4         ImGui::EndTabItem();
5     }
6     if (ImGui::BeginTabItem("Ag")) {
7         ImGui::Text("Ag ayarlari");
8         ImGui::EndTabItem();
9     }
10    if (ImGui::BeginTabItem("Hakkinda")) {
11        ImGui::Text("Surum 1.0");
12        ImGui::EndTabItem();
13    }
14    ImGui::EndTabBar();
15 }

```

10.6 Açılır Pencereler ve Modal'lar (Popup)

```

1 // Sag tik menu:
2 if (ImGui::Button("Sag-tik menu ac")) ImGui::OpenPopup("baglam");
3 if (ImGui::BeginPopup("baglam")) {
4     if (ImGui::MenuItem("Kopyala")) {}
5     if (ImGui::MenuItem("Yapistir")) {}
6     ImGui::EndPopup();
7 }
8
9 // Modal onay penceresi:
10 if (ImGui::Button("Sil")) ImGui::OpenPopup("Emin misiniz?");
11 if (ImGui::BeginPopupModal("Emin misiniz?", nullptr,
12     ImGuiWindowFlags_AlwaysAutoResize)) {
13     ImGui::Text("Bu islem geri alinamaz.");
14     if (ImGui::Button("Evet", ImVec2(120,0))) {
15         do_delete();
16         ImGui::CloseCurrentPopup();
17     }
18     ImGui::SameLine();
19     if (ImGui::Button("Hayir", ImVec2(120,0))) ImGui::CloseCurrentPopup();
20     ImGui::EndPopup();
21 }

```

10.7 İpucu Balonları (Tooltip)

```

1 ImGui::Text("Uzerine gel (?)");
2 if (ImGui::IsItemHovered()) {
3     ImGui::SetTooltip("Bu bir yardim balonudur.");
4 }
5 // Zengin icerikli tooltip:
6 ImGui::Button("Detay");
7 if (ImGui::IsItemHovered()) {
8     ImGui::BeginTooltip();
9     ImGui::Text("Baslik");
10    ImGui::Separator();
11    ImGui::BulletText("Satir 1");
12    ImGui::BulletText("Satir 2");
13    ImGui::EndTooltip();
14 }

```

11. Tablolar: Güçlü Veri Gösterimi

Table API'si ImGui'nin en yeteneklidir: sıralama, yeniden boyutlandırma, dondurma (freeze), sütun gizleme ve kaydırma destekler. Elektronik tablo benzeri araçların temelidir.

11.1 Temel Tablo

```

1 ImGuiTableFlags flags = ImGuiTableFlags_Borders
2                       | ImGuiTableFlags_RowBg
3                       | ImGuiTableFlags_Resizable;
4 if (ImGui::BeginTable("kisiler", 3, flags)) {
5     // Baslik satiri
6     ImGui::TableSetupColumn("Ad");

```

```

7   ImGui::TableSetupColumn("Yas");
8   ImGui::TableSetupColumn("Sehir");
9   ImGui::TableHeadersRow();
10
11  struct Row { const char* ad; int yas; const char* sehir; };
12  Row rows[] = {{"Ada",30,"Izmir"}, {"Kerem",25,"Ankara"}, {"Zeynep",28,"Bursa"}};
13  for (auto& r : rows) {
14      ImGui::TableNextRow();
15      ImGui::TableSetColumnIndex(0); ImGui::Text("%s", r.ad);
16      ImGui::TableSetColumnIndex(1); ImGui::Text("%d", r.yas);
17      ImGui::TableSetColumnIndex(2); ImGui::Text("%s", r.sehir);
18  }
19  ImGui::EndTable();
20 }

```

11.2 Sıralanabilir ve Dondurulmuş Başlıklı Tablo

```

1  ImGuiTableFlags flags = ImGuiTableFlags_Borders | ImGuiTableFlags_RowBg
2                        | ImGuiTableFlags_Sortable
3                        | ImGuiTableFlags_ScrollY;
4  if (ImGui::BeginTable("veri", 3, flags, ImVec2(0, 200))) {
5      ImGui::TableSetupScrollFreeze(0, 1); // ilk satiri dondur (kayarken sabit)
6      ImGui::TableSetupColumn("ID", ImGuiTableColumnFlags_DefaultSort);
7      ImGui::TableSetupColumn("Ad");
8      ImGui::TableSetupColumn("Puan");
9      ImGui::TableHeadersRow();
10
11     // ImGuiTableSortSpecs* specs = ImGui::TableGetSortSpecs();
12     // -> specs->Specs[0].ColumnIndex ve SortDirection ile verinizi siralayin
13
14     for (int i = 0; i < 100; i++) {
15         ImGui::TableNextRow();
16         ImGui::TableNextColumn(); ImGui::Text("%d", i);
17         ImGui::TableNextColumn(); ImGui::Text("Oge %d", i);
18         ImGui::TableNextColumn(); ImGui::Text("%d", (i * 37) % 100);
19     }
20     ImGui::EndTable();
21 }

```

İpucu

Çok satırlı tablolarda (binlerce satır) her satırı çizmek yerine `ImGuiListClipper` kullanın: sadece görünür satırlar çizilir. Kısım 3'te detaylandırıyoruz. Table + Clipper birlikte, milyonlarca satırlık veriyi akıcı gösterebilir.

12. Grafikler ve Görselleştirme

ImGui çekirdeği basit çizgi/histogram grafikleri sunar. Daha ciddi görselleştirme için `ImPlot` eklentisi kullanılır, ama çekirdek araçlar hızlı hata ayıklama için yeterlidir.

12.1 Çizgi ve Histogram

```

1  float samples[90];
2  for (int i = 0; i < 90; i++)

```

```
3     samples[i] = sinf(i * 0.2f + ImGui::GetTime());
4
5     ImGui::PlotLines("Sinus", samples, IM_ARRAYSIZE(samples),
6                     0, "canli", -1.0f, 1.0f, ImVec2(0, 80));
7
8     float bars[] = {0.2f, 0.7f, 0.4f, 0.9f, 0.5f, 0.3f};
9     ImGui::PlotHistogram("Cubuklar", bars, IM_ARRAYSIZE(bars),
10                        0, nullptr, 0.0f, 1.0f, ImVec2(0, 80));
```

12.2 İlerleme Çubuğu

```
1 static float progress = 0.0f;
2 progress += ImGui::GetIO().DeltaTime * 0.3f;
3 if (progress > 1.0f) progress = 0.0f;
4
5 ImGui::ProgressBar(progress, ImVec2(-FLT_MIN, 0));
6 ImGui::ProgressBar(progress, ImVec2(-FLT_MIN, 0), "Yukleniyor...");
```

ImPlot ne zaman?

Kaydırılabilir/yakınlaştırılabilir eksenler, birden çok seri, gerçek zamanlı kayan grafik, saçılım (scatter), ısı haritası istiyorsanız ImPlot (<https://github.com/epezent/implot>) kullanın. Çekirdeğin PlotLines'ı yalnızca “şu diziye hızlıca bir göz atayım” durumları içindir.

KISIM 3

İleri Konular

13. Stil ve Tema

ImGui'nin görünümü ImGuiStyle yapısı üzerinden tamamen özelleştirilir. Renkler, kenar yuvarlaklığı, boşluklar, kenarlık kalınlıkları hepsi buradadır.

13.1 Hazır Temalar ve Genel Ayarlar

```

1 ImGui::StyleColorsDark();    // koyu (varsayılan)
2 ImGui::StyleColorsLight();   // acik
3 ImGui::StyleColorsClassic(); // eski klasik
4
5 ImGuiStyle& style = ImGui::GetStyle();
6 style.WindowRounding        = 6.0f;    // pencere koseleri
7 style.FrameRounding         = 4.0f;    // dugme/kutu koseleri
8 style.GrabRounding          = 4.0f;    // slider tutamaci
9 style.WindowPadding         = ImVec2(10, 10);
10 style.ItemSpacing           = ImVec2(8, 6);
11 style.ScrollbarSize         = 14.0f;
12 style.WindowBorderSize      = 1.0f;

```

13.2 Renkleri Elle Ayarlamak

```

1 ImVec4* colors = ImGui::GetStyle().Colors;
2 colors[ImGuiCol_WindowBg]      = ImVec4(0.10f, 0.10f, 0.12f, 1.00f);
3 colors[ImGuiCol_Button]       = ImVec4(0.20f, 0.45f, 0.75f, 1.00f);
4 colors[ImGuiCol_ButtonHovered] = ImVec4(0.26f, 0.55f, 0.90f, 1.00f);
5 colors[ImGuiCol_ButtonActive]  = ImVec4(0.15f, 0.38f, 0.65f, 1.00f);
6 colors[ImGuiCol_Header]       = ImVec4(0.20f, 0.45f, 0.75f, 0.55f);
7 colors[ImGuiCol_FrameBg]      = ImVec4(0.16f, 0.16f, 0.20f, 1.00f);
8 colors[ImGuiCol_Text]         = ImVec4(0.90f, 0.90f, 0.92f, 1.00f);

```

13.3 Yerel (Geçici) Stil Değişikliği: Push/Pop

Belirli bir widget için stili geçici değiştirmek istersek Push/Pop kullanırız. **Her Push bir Pop ile eşleşmelidir**, yoksa stil yığını bozulur.

```

1 // Sadece bu dugme kirmizi olsun:
2 ImGui::PushStyleColor(ImGuiCol_Button,      ImVec4(0.7f,0.15f,0.15f,1));
3 ImGui::PushStyleColor(ImGuiCol_ButtonHovered, ImVec4(0.85f,0.2f,0.2f,1));

```



```

4 if (ImGui::Button("TEHLIKELI: Sil")) { }
5 ImGui::PopStyleColor(2);    // 2 renk push edildi -> 2 pop
6
7 // Gecici deger degisikligi:
8 ImGui::PushStyleVar(ImGuiStyleVar_FrameRounding, 12.0f);
9 ImGui::Button("Yuvarlak dugme");
10 ImGui::PopStyleVar();

```

Dikkat

Push/Pop sayıları tutmazsa hata ID sisteminden bile kötüdür: stil yığını kareler arası taşar ve tüm arayüz bozulur ya da assert atar. Kural: her PushStyleColor(n renk) için PopStyleColor(n). Bir kod yolunda return varsa Pop'u atlamamaya dikkat edin (RAII sarıcı yazmak iyi fikir).

14. Fontlar ve Türkçe Karakter Desteği

ImGui'nin varsayılan fontu yalnızca temel ASCII içerir; Türkçe “ç, ğ, ı, İ, ş, ö, ü” karakterleri görünmez. Bunu çözmek için bir TTF font yükleyip doğru *glyph aralığını* belirtmeliyiz.

14.1 Türkçe Fontu Yükleme

```

1 ImGuiIO& io = ImGui::GetIO();
2
3 // Turkce dahil Latin karakterleri iceren glyph araligi:
4 static const ImWchar ranges[] = {
5     0x0020, 0x00FF,    // Temel Latin + Latin-1 (aksanli harfler)
6     0x0100, 0x017F,    // Latin Extended-A (s-cedilla, g-breve: s, g)
7     0,
8 };
9 ImFontConfig cfg;
10 cfg.OversampleH = 2;
11 cfg.OversampleV = 1;
12
13 io.Fonts->AddFontFromFileTTF(
14     "fonts/DejaVuSans.ttf", 18.0f, &cfg, ranges);
15
16 // Font atlası backend baslatılmadan ONCE olusturulmalı; genelde
17 // ImGui_ImplOpenGL3_Init otomatik yapar. Elle: io.Fonts->Build();

```

Bilgi

Türkçe için kritik karakterler: ğ (U+011F), Ğ (U+011E), ş (U+015F), Ş (U+015E), ı (U+0131), İ (U+0130). Bunlar Latin Extended-A (0x0100–0x017F) aralığındadır. ç, ö, ü ise Latin-1'de (0x00FF'e kadar) yer alır. İkisini de eklemeyerseniz kelimeler eksik görünür. Kod dosyanız da UTF-8 kaydedilmeli.

14.2 Birden Fazla Font ve Simge Fontları

```

1 ImFont* font_default = io.Fonts->AddFontDefault();
2 ImFont* font_big = io.Fonts->AddFontFromFileTTF("fonts/Roboto.ttf", 32.0f);
3
4 // Kullanırken:
5 ImGui::PushFont(font_big);

```

```

6 ImGui::Text("BUYUK BASLIK");
7 ImGui::PopFont();
8
9 // Ikon fontlarini (FontAwesome) metin fontuyla birlestirmek:
10 ImFontConfig icons_cfg;
11 icons_cfg.MergeMode = true; // oncesi fontun uzerine birlestir
12 static const ImWchar icon_ranges[] = { 0xf000, 0xf3ff, 0 };
13 io.Fonts->AddFontFromFileTTF("fonts/fa-solid.ttf", 16.0f,
14                               &icons_cfg, icon_ranges);

```

15. Docking ve Çoklu Viewport

Docking, pencereleri birbirine ve ana pencerenin kenarlarına yapıştırma özelliğidir — IDE benzeri düzenler yaratır. Viewport ise ImGui pencerelerini işletim sistemi penceresi olarak ana pencerenin dışına taşımaya izin verir. İkisi de ImGui'nin docking dalında bulunur.

15.1 Docking'i Etkinleştirme

```

1 ImGuiIO& io = ImGui::GetIO();
2 io.ConfigFlags |= ImGuiConfigFlags_DockingEnable; // docking
3 io.ConfigFlags |= ImGuiConfigFlags_ViewportsEnable; // dis pencereler
4
5 // Viewport acikken render dongusunun sonuna eklenir:
6 ImGui::Render();
7 // ... normal cizim ...
8 if (io.ConfigFlags & ImGuiConfigFlags_ViewportsEnable) {
9     GLFWwindow* backup = glfwGetCurrentContext();
10    ImGui::UpdatePlatformWindows();
11    ImGui::RenderPlatformWindowsDefault();
12    glfwMakeContextCurrent(backup);
13 }

```

15.2 Ana DockSpace Oluşturma

Tüm ekranı kaplayan, diğer pencerelerin yapışacağı bir dock alanı:

```

1 void DrawDockSpace() {
2     ImGuiViewport* vp = ImGui::GetMainViewport();
3     ImGui::SetNextWindowPos(vp->WorkPos);
4     ImGui::SetNextWindowSize(vp->WorkSize);
5     ImGui::SetNextWindowViewport(vp->ID);
6
7     ImGuiWindowFlags flags = ImGuiWindowFlags_NoTitleBar
8         | ImGuiWindowFlags_NoResize | ImGuiWindowFlags_NoMove
9         | ImGuiWindowFlags_NoBringToFrontOnFocus | ImGuiWindowFlags_NoNavFocus
10        | ImGuiWindowFlags_MenuBar;
11
12    ImGui::PushStyleVar(ImGuiStyleVar_WindowRounding, 0.0f);
13    ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
14    ImGui::Begin("DockSpaceRoot", nullptr, flags);
15    ImGui::PopStyleVar(2);
16
17    ImGuiID dock_id = ImGui::GetID("MainDockSpace");
18    ImGui::DockSpace(dock_id, ImVec2(0, 0), ImGuiDockNodeFlags_None);
19 }

```

```

20     ImGui::End();
21 }

```

docking dalı

Docking ve viewport, ImGui'nin docking branch'inde yer alır (ana master'da değil). FetchContent kullanıyorsanız GIT_TAG'i docking yapın veya docking özellikli bir sürüm etiketi seçin. API'ler kararludur ve üretimde yaygın kullanılır.

16. DrawList ile Özel Çizim

Widget'lar yetmediğinde ImDrawList ile doğrudan çizgi, dikdörtgen, daire, metin ve dolgu çizebilirsiniz. Bu, özel grafikler, düğüm editörleri, oyun mini haritaları ve tuval uygulamalarının temelidir.

16.1 DrawList'e Erişim

```

1 // Mevcut pencerenin çizim listesi (widget'ların arkası/onu):
2 ImDrawList* dl = ImGui::GetWindowDrawList();
3
4 // Tüm ekranın üstüne çizmek için (overlay):
5 ImDrawList* fg = ImGui::GetForegroundDrawList();
6 ImDrawList* bg = ImGui::GetBackgroundDrawList();

```

16.2 Temel Şekiller

Koordinatlar **ekran uzayındadır** (pencere değil). Genelde bir taban nokta alıp ona göre çizeriz.

```

1 ImVec2 p = ImGui::GetCursorScreenPos(); // çizim başlangıcı (ekran uzayı)
2 ImU32 white = IM_COL32(255,255,255,255);
3 ImU32 blue = IM_COL32( 60,140,230,255);
4
5 dl->AddLine(p, ImVec2(p.x+100, p.y+50), white, 2.0f);
6 dl->AddRect(ImVec2(p.x, p.y), ImVec2(p.x+80, p.y+80), white, 4.0f, 0, 1.5f);
7 dl->AddRectFilled(ImVec2(p.x+90,p.y), ImVec2(p.x+170,p.y+80), blue, 4.0f);
8 dl->AddCircle(ImVec2(p.x+220,p.y+40), 40, white, 32, 2.0f);
9 dl->AddCircleFilled(ImVec2(p.x+320,p.y+40), 40, blue, 32);
10 dl->AddTriangleFilled(ImVec2(p.x,p.y+120), ImVec2(p.x+60,p.y+120),
11                     ImVec2(p.x+30,p.y+90), white);
12 dl->AddText(ImVec2(p.x, p.y+140), white, "DrawList metni");
13 dl->AddBezierCubic(p, ImVec2(p.x+50,p.y-40), ImVec2(p.x+100,p.y+80),
14                 ImVec2(p.x+150,p.y), blue, 2.0f);
15
16 // Çizdiğimiz alanın yer kaplaması için bir kukla oge bırak:
17 ImGui::Dummy(ImVec2(360, 180));

```

16.3 Etkileşimli Tuval Deseni

Çizim alanını tıklanabilir yapmak için görünmez bir buton (InvisibleButton) kullanılır; sonra fare konumunu alıp çizeriz.

```

1 ImVec2 canvas_p0 = ImGui::GetCursorScreenPos();
2 ImVec2 canvas_sz = ImGui::GetContentRegionAvail();
3 ImVec2 canvas_p1 = ImVec2(canvas_p0.x + canvas_sz.x,
4                             canvas_p0.y + canvas_sz.y);

```

```

5
6 ImGuiDrawList* dl = ImGui::GetWindowDrawList();
7 dl->AddRectFilled(canvas_p0, canvas_p1, IM_COL32(30,30,35,255));
8
9 ImGui::InvisibleButton("canvas", canvas_sz);
10 bool hovered = ImGui::IsItemHovered();
11 bool active = ImGui::IsItemActive();
12
13 if (active) {
14     ImVec2 m = ImGui::GetIO().MousePos;
15     dl->AddCircleFilled(m, 5, IM_COL32(255,180,0,255));
16 }

```

İpucu

DrawList'e çizerken PushClipRect/PopClipRect ile çizimi bir dikdörtgene sınırlayabilirsiniz — tuval dışına taşmayı engeller. Ayrıca dl->ChannelsSplit/ChannelsMerge ile çizim katmanlarını (örneğin düğümlerin arkasındaki bağlantı çizimleri) yönetebilirsiniz.

17. InputText Geri Çağırımları (Callback)

InputText'in davranışını, bir callback vererek özelleştirebiliriz: otomatik tamamlama, geçmiş (history), karakter filtreleme veya her değişimde tetikleme.

17.1 Callback Bayrakları ve İskelet

```

1 struct InputState { std::vector<std::string> history; int pos = -1; };
2
3 int TextCallback(ImGuiInputTextCallbackData* data) {
4     InputState* st = (InputState*)data->UserData;
5     switch (data->EventFlag) {
6     case ImGuiInputTextFlags_CallbackCompletion:
7         // TAB'a basıldı: otomatik tamamlama yap
8         data->InsertChars(data->CursorPos, "_tamamlandi");
9         break;
10    case ImGuiInputTextFlags_CallbackHistory:
11        // Yukari/asagi ok: gecmiste gez
12        if (data->EventKey == ImGuiKey_UpArrow && st->pos + 1 < (int)st->history.size()) {
13            st->pos++;
14            data->DeleteChars(0, data->BufTextLen);
15            data->InsertChars(0, st->history[st->pos].c_str());
16        }
17        break;
18    case ImGuiInputTextFlags_CallbackCharFilter:
19        // Sadece rakam kabul et:
20        if (data->EventChar < '0' || data->EventChar > '9') return 1; // reddet
21        break;
22    }
23    return 0;
24 }
25
26 // Kullanım:
27 static char buf[256] = "";
28 static InputState state;
29 ImGuiInputTextFlags flags = ImGuiInputTextFlags_CallbackCompletion

```

```

30         | ImGuiInputTextFlags_CallbackHistory
31         | ImGuiInputTextFlags_EnterReturnsTrue;
32 if (ImGui::InputText("Komut", buf, sizeof(buf), flags, TextCallback, &state)) {
33     // Enter'a basıldı
34     state.history.insert(state.history.begin(), buf);
35     state.pos = -1;
36     buf[0] = '\0';
37 }

```

18. Klavye, Fare ve Öge Durumu Sorgulama

ImGui, en son çizilen öge hakkında zengin sorgu fonksiyonları sunar. Bunlar etkileşimli arayüzlerin kalbidir.

18.1 Öge Durumu Fonksiyonları

```

1 ImGui::Button("Test");
2 // Bir önceki widget hakkında:
3 bool hovered = ImGui::IsItemHovered(); // fare üzerinde mi
4 bool clicked = ImGui::IsItemClicked(); // bu kare tıklandı mı
5 bool active = ImGui::IsItemActive(); // basılı tutuluyor mu
6 bool focused = ImGui::IsItemFocused(); // klavye odakında mı
7 bool edited = ImGui::IsItemEdited(); // değeri bu kare degisti mi
8 ImVec2 rmin = ImGui::GetItemRectMin(); // ogenin sinirlari
9 ImVec2 rmax = ImGui::GetItemRectMax();
10
11 // Pencere hakkında:
12 bool win_hovered = ImGui::IsWindowHovered();
13 bool win_focused = ImGui::IsWindowFocused();

```

18.2 Doğrudan Klavye ve Fare

```

1 ImGuiIO& io = ImGui::GetIO();
2
3 // Fare:
4 ImVec2 mouse = io.MousePos;
5 bool left_down = ImGui::IsMouseDown(ImGuiMouseButton_Left);
6 bool right_click = ImGui::IsMouseClicked(ImGuiMouseButton_Right);
7 float wheel = io.MouseWheel; // scroll miktarı
8 ImVec2 drag = ImGui::GetMouseDragDelta(ImGuiMouseButton_Left);
9
10 // Klavye (yeni API):
11 if (ImGui::IsKeyPressed(ImGuiKey_Space)) { jump(); }
12 if (ImGui::IsKeyDown(ImGuiKey_W)) { move_forward(); }
13 if (ImGui::IsKeyChordPressed(ImGuiMod_Ctrl | ImGuiKey_S)) { save(); }

```

Dikkat

Oyun/uygulama girdisiyle ImGui girdisi çakışabilir. `io.WantCaptureMouse` ve `io.WantCaptureKeyboard` bayraklarını kontrol edin: bunlar true iken girdi ImGui'ye aittir (örn. kullanıcı bir InputText'e yazıyor), oyununuz o girdiyi *yok saymalıdır*. Aksi hâlde metin yazarken karakter hareket eder.

19. Performans: ListClipper ve İpuçları

Immediate mode her kare tüm UI'yı çalıştırır. Çoğu araç için bu sorunsuzdur, ama binlerce satırlık listelerde görünmeyen öğeleri çizmek israftır. ImGuiListClipper yalnızca görünür aralığı çizer.

19.1 ListClipper Kullanımı

```
1 // 1.000.000 satırlık liste -- yalnızca gorunenler cizilir
2 ImGuiListClipper clipper;
3 clipper.Begin(1000000);
4 while (clipper.Step()) {
5     for (int i = clipper.DisplayStart; i < clipper.DisplayEnd; i++) {
6         ImGui::Text("Satir %d: veri...", i);
7     }
8 }
```

19.2 Genel Performans İpuçları

İpucu

Neden

Ağır işi UI'dan ayır	Dosya/ağ/hesap işini UI kodunda yapma; sonucu göster.
ListClipper kullan	Uzun listelerde görünmeyeni çizme.
Gereksiz sprintf azalt	Her karede binlerce format çağrısı birikir.
Kapalı pencereyi erken çık	if(!Begin(...)){End();return;} ile içeriği atla.
Clip'i backend'e bırak	ImGui zaten görünmeyen pencereyi çizmez.

Bilgi

ImGui'nin kendisi neredeyse hiç yavaşlamaz — darboğaz genelde *sizin* UI kodunuzun her kare yaptığı gereksiz hesaplardır. Şüpheye düşerseniz ImGui::ShowMetricsWindow() penceresini açın: kaç vertex, kaç pencere, kaç çizim çağrısı ürettiğinizi canlı gösterir.

19.3 Yerleşik Demo ve Yardımcı Pencereleler

```
1 static bool show_demo = true;
2 if (show_demo) ImGui::ShowDemoWindow(&show_demo); // her seyi gosteren demo
3 ImGui::ShowMetricsWindow(nullptr); // performans/ic durum
4 ImGui::ShowStyleEditor(); // canli stil duzenleyici
5 ImGui::ShowUserGuide(); // kisayol rehberi
```

İpucu

ShowDemoWindow bir öğrenme aracından çok daha fazlasıdır: yeni bir widget'ı nasıl kullanacağınızı unuttuğunuzda demo penceresini açın, ilgili bölümü bulun ve sağ üstteki menüden "item picker" ile o widget'ın imgui_demo.cpp'deki kaynak satırına gidin.

KISIM 4

Projeler

Bu kısımda öğrendiğimiz her şeyi **tam, çalışan projelere** dönüştürüyoruz. Önce tüm projelerin paylaşacağı yeniden kullanılabilir bir uygulama iskeleti kuruyoruz; sonra her proje yalnızca bir “çiz” fonksiyonu yazarak bu iskelete bağlanıyor. Projeler basitten karmaşığa doğru sıralanmıştır.

20. Proje 0: Yeniden Kullanılabilir Uygulama İskeleti

Kısım 1'deki `main.cpp`'yi her seferinde tekrar yazmamak için pencere kurulumunu, ana döngüyü ve temizliği bir sınıfa saralım. Bundan sonraki tüm projeler sadece `OnFrame()` fonksiyonunu doldurur.

20.1 app.h — İskelet

```

1 // app.h -- tum projelerin ortak temeli
2 #pragma once
3 #include "imgui.h"
4 #include "imgui_impl_glfw.h"
5 #include "imgui_impl_opengl3.h"
6 #include <GLFW/glfw3.h>
7 #include <functional>
8 #include <cstdio>
9
10 class App {
11 public:
12     App(const char* title, int w = 1280, int h = 720) {
13         glfwSetErrorCallback([](int e, const char* d){
14             std::fprintf(stderr, "GLFW %d: %s\n", e, d); });
15         glfwInit();
16         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
17         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
18         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
19         window_ = glfwCreateWindow(w, h, title, nullptr, nullptr);
20         glfwMakeContextCurrent(window_);
21         glfwSwapInterval(1);
22
23         IMGUI_CHECKVERSION();
24         ImGui::CreateContext();
25         ImGui::GetIO().ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
26         ImGui::StyleColorsDark();
27         ImGui_ImplGlfw_InitForOpenGL(window_, true);
28         ImGui_ImplOpenGL3_Init("#version 330");
29     }
30 
```

```

31 ~App() {
32     ImGui_ImplOpenGL3_Shutdown();
33     ImGui_ImplGlfw_Shutdown();
34     ImGui::DestroyContext();
35     glfwDestroyWindow(window_);
36     glfwTerminate();
37 }
38
39 // Her kare cagirilacak cizim fonksiyonunu alir ve dongusu surer.
40 void Run(const std::function<void()>& on_frame) {
41     while (!glfwWindowShouldClose(window_)) {
42         glfwPollEvents();
43         ImGui_ImplOpenGL3_NewFrame();
44         ImGui_ImplGlfw_NewFrame();
45         ImGui::NewFrame();
46
47         on_frame(); // <-- projenin UI kodu
48
49         ImGui::Render();
50         int dw, dh;
51         glfwGetFramebufferSize(window_, &dw, &dh);
52         glViewport(0, 0, dw, dh);
53         glClearColor(0.09f, 0.10f, 0.12f, 1.0f);
54         glClear(GL_COLOR_BUFFER_BIT);
55         ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
56         glfwSwapBuffers(window_);
57     }
58 }
59
60 private:
61     GLFWwindow* window_ = nullptr;
62 };

```

20.2 Kullanım Kalıbı

Bundan sonra her proje bu iki satırla başlar; biz sadece köşeli parantezin içini doldururuz:

```

1 #include "app.h"
2 int main() {
3     App app("Proje Adi");
4     app.Run([&]() {
5         // ... bu projenin her kare cizilen UI kodu ...
6     });
7 }

```

21. Proje 1: Yapılacaklar (To-Do) Uygulaması

İlk projemiz klasik bir “yapılacaklar listesi”. Ekleme, tamamlama olarak işaretleme, silme ve filtreleme yapar. Immediate mode’un veri-yansıtma felsefesini net gösterir: UI, tasks vektörünün her karede çizilen görüntüsüdür.

21.1 Tam Kaynak

```

1 #include "app.h"
2 #include <vector>

```



```

3  #include <string>
4
5  struct Task {
6      std::string text;
7      bool done = false;
8  };
9
10 int main() {
11     App app("Yapılacaklar");
12     std::vector<Task> tasks;
13     char input[256] = "";
14     int filter = 0;    // 0: tumu, 1: aktif, 2: tamamlanan
15
16     app.Run([&]() {
17         ImGui::SetNextWindowPos(ImVec2(0,0), ImGuiCond_Once);
18         ImGui::SetNextWindowSize(ImGui::GetMainViewport()->Size, ImGuiCond_Once);
19         ImGui::Begin("Gorevlerim", nullptr, ImGuiWindowFlags_NoResize
20                     | ImGuiWindowFlags_NoMove | ImGuiWindowFlags_NoCollapse);
21
22         // --- Ekleme satiri ---
23         ImGui::SetNextItemWidth(-100);
24         bool enter = ImGui::InputText("##yeni", input, sizeof(input),
25                                     ImGuiInputTextFlags_EnterReturnsTrue);
26         ImGui::SameLine();
27         bool add = ImGui::Button("Ekle", ImVec2(-FLT_MIN, 0));
28         if ((enter || add) && input[0] != '\0') {
29             tasks.push_back({input, false});
30             input[0] = '\0';
31         }
32
33         // --- Filtre ---
34         ImGui::RadioButton("Tumu", &filter, 0);    ImGui::SameLine();
35         ImGui::RadioButton("Aktif", &filter, 1);    ImGui::SameLine();
36         ImGui::RadioButton("Biten", &filter, 2);
37         ImGui::Separator();
38
39         // --- Liste ---
40         int to_delete = -1;
41         for (int i = 0; i < (int)tasks.size(); i++) {
42             Task& t = tasks[i];
43             if (filter == 1 && t.done) continue;
44             if (filter == 2 && !t.done) continue;
45
46             ImGui::PushID(i);                                // benzersiz ID!
47             ImGui::Checkbox("##done", &t.done);
48             ImGui::SameLine();
49             if (t.done) {
50                 ImGui::PushStyleColor(ImGuiCol_Text, ImVec4(0.5f,0.5f,0.5f,1));
51                 ImGui::Text("%s", t.text.c_str());
52                 ImGui::PopStyleColor();
53             } else {
54                 ImGui::Text("%s", t.text.c_str());
55             }
56             ImGui::SameLine(ImGui::GetWindowWidth() - 50);
57             if (ImGui::SmallButton("Sil")) to_delete = i;
58             ImGui::PopID();
59         }
60         if (to_delete >= 0) tasks.erase(tasks.begin() + to_delete);
61     });

```

```

62 // --- Özet ---
63 int done = 0; for (auto& t : tasks) if (t.done) done++;
64 ImGui::Separator();
65 ImGui::Text("Toplam: %d | Biten: %d | Kalan: %d",
66             (int)tasks.size(), done, (int)tasks.size() - done);
67
68 ImGui::End();
69 });
70 }

```

Neyi öğrendik?

Silmeyi döngü *içinde* yapmayıp `to_delete` değişkenine erteledik — çünkü çizim sürerken vektörü değiştirmek indeksleri bozar. Bu, immediate mode listelerinde standart bir kalıptır: “döngüde niyeti kaydet, döngüden sonra uygula”. Ayrıca her satırda `PushID(i)` ile ID çakışmasını önledik.

22. Proje 2: Renkli Log Konsolu

Hemen her araçta bir log/konsol penceresi gerekir. Bu proje; seviyeye göre renklendirme, metin filtresi, otomatik aşağı kaydırma ve komut girişi olan yeniden kullanılabilir bir konsol sınıfı sunar.

22.1 Tam Kaynak

```

1  #include "app.h"
2  #include <vector>
3  #include <string>
4  #include <cstdarg>
5
6  enum LogLevel { LOG_INFO, LOG_WARN, LOG_ERROR };
7
8  struct Console {
9      struct Line { LogLevel level; std::string text; };
10     std::vector<Line> lines;
11     char input[256] = "";
12     ImGuiTextFilter filter;
13     bool auto_scroll = true;
14     bool show_info = true, show_warn = true, show_error = true;
15
16     void Add(LogLevel lv, const char* fmt, ...) {
17         char buf[1024];
18         va_list args; va_start(args, fmt);
19         vsnprintf(buf, sizeof(buf), fmt, args);
20         va_end(args);
21         lines.push_back({lv, buf});
22     }
23
24     void Draw(const char* title) {
25         ImGui::Begin(title);
26
27         // Ust arac cubugu
28         if (ImGui::Button("Temizle")) lines.clear();
29         ImGui::SameLine(); ImGui::Checkbox("Info", &show_info);
30         ImGui::SameLine(); ImGui::Checkbox("Uyari", &show_warn);
31         ImGui::SameLine(); ImGui::Checkbox("Hata", &show_error);
32         ImGui::SameLine(); ImGui::Checkbox("Oto-kaydir", &auto_scroll);

```

```

33     filter.Draw("Filtre", 180);
34     ImGui::Separator();
35
36     // Kaydirilabilir log alani (giris satirina yer birak)
37     float footer = ImGui::GetFrameHeightWithSpacing();
38     ImGui::BeginChild("scroll", ImVec2(0, -footer), false,
39         ImGuiWindowFlags_HorizontalScrollbar);
40     for (auto& ln : lines) {
41         if (ln.level == LOG_INFO && !show_info) continue;
42         if (ln.level == LOG_WARN && !show_warn) continue;
43         if (ln.level == LOG_ERROR && !show_error) continue;
44         if (!filter.PassFilter(ln.text.c_str())) continue;
45
46         ImVec4 col(0.85f,0.85f,0.85f,1);
47         if (ln.level == LOG_WARN) col = ImVec4(1.0f,0.8f,0.2f,1);
48         if (ln.level == LOG_ERROR) col = ImVec4(1.0f,0.4f,0.4f,1);
49         ImGui::PushStyleColor(ImGuiCol_Text, col);
50         ImGui::TextUnformatted(ln.text.c_str());
51         ImGui::PopStyleColor();
52     }
53     if (auto_scroll && ImGui::GetScrollY() >= ImGui::GetScrollMaxY())
54         ImGui::SetScrollHereY(1.0f);
55     ImGui::EndChild();
56
57     ImGui::Separator();
58     // Komut girisi
59     ImGui::SetNextItemWidth(-60);
60     bool run = ImGui::InputText("##cmd", input, sizeof(input),
61         ImGuiInputTextFlags_EnterReturnsTrue);
62     ImGui::SameLine();
63     run |= ImGui::Button("Gonder");
64     if (run && input[0]) {
65         Add(LOG_INFO, "> %s", input);
66         input[0] = '\0';
67         ImGui::SetKeyboardFocusHere(-1); // odagi girise geri ver
68     }
69     ImGui::End();
70 }
71 };
72
73 int main() {
74     App app("Log Konsolu");
75     Console console;
76     console.Add(LOG_INFO, "Sistem baslatildi.");
77     console.Add(LOG_WARN, "Yapilandirma dosyasi bulunamadi, varsayilan kullaniliyor.");
78     console.Add(LOG_ERROR, "Sunucuya baglanilamadi (port 8080).");
79
80     float timer = 0;
81     app.Run([&]() {
82         // Her saniye otomatik bir log ekle (demo)
83         timer += ImGui::GetIO().DeltaTime;
84         if (timer > 1.0f) {
85             timer = 0;
86             console.Add(LOG_INFO, "Kalp atisi: t=%.1f", ImGui::GetTime());
87         }
88         console.Draw("Konsol");
89     });
90 }

```

Bilgi

ImGuiTextFilter, ImGui'nin yerleşik filtre yardımcısıdır: "hata,-baglanti" gibi ifadeleri anlar ("hata içeren ama baglanti içermeyen"). `filter.Draw()` bir arama kutusu çizer, `filter.PassFilter(metin)` her satır için testi yapar. Konsollar, listeler ve tablolarda çok işe yarar.

23. Proje 3: Canlı Sistem Monitörü

Bu proje kayan (scrolling) grafikler çizer: sahte CPU/RAM değerlerini bir halka tampona (ring buffer) yazıp `PlotLines` ile gösterir. Gerçek bir uygulamada bu verileri işletim sisteminden okurdunuz; burada mantığa odaklanıyoruz.

23.1 Kayan Tampon (Scrolling Buffer)

```

1  #include "app.h"
2  #include <cmath>
3
4  struct ScrollBuffer {
5      static const int SIZE = 120;
6      float data[SIZE] = {};
7      int offset = 0;
8
9      void Push(float v) { data[offset] = v; offset = (offset + 1) % SIZE; }
10     float Max() const {
11         float m = 0; for (float v : data) if (v > m) m = v; return m;
12     }
13     float Last() const { return data[(offset - 1 + SIZE) % SIZE]; }
14 };
15
16 int main() {
17     App app("Sistem Monitoru");
18     ScrollBuffer cpu, ram;
19     float t = 0;
20
21     app.Run([&]() {
22         // Sahte veri uret
23         t += ImGui::GetIO().DeltaTime;
24         cpu.Push(50 + 40 * sinf(t * 1.5f) + 10 * sinf(t * 7.0f));
25         ram.Push(60 + 20 * sinf(t * 0.4f));
26
27         ImGui::Begin("Kaynak Kullanimi");
28
29         // --- CPU ---
30         ImGui::Text("CPU: %.1f%%", cpu.Last());
31         ImGui::PushStyleColor(ImGuiCol_PlotLines, ImVec4(0.2f,0.8f,0.4f,1));
32         ImGui::PlotLines("##cpu", cpu.data, ScrollBuffer::SIZE, cpu.offset,
33                         nullptr, 0.0f, 100.0f, ImVec2(-FLT_MIN, 80));
34         ImGui::PopStyleColor();
35
36         // --- RAM ---
37         ImGui::Text("RAM: %.1f%%", ram.Last());
38         ImGui::PushStyleColor(ImGuiCol_PlotLines, ImVec4(0.3f,0.6f,1.0f,1));
39         ImGui::PlotLines("##ram", ram.data, ScrollBuffer::SIZE, ram.offset,
40                         nullptr, 0.0f, 100.0f, ImVec2(-FLT_MIN, 80));
41         ImGui::PopStyleColor();

```

```

42     ImGui::Separator();
43     // Özet tablo
44     if (ImGui::BeginTable("özet", 3, ImGuiTableFlags_Borders)) {
45         ImGui::TableSetupColumn("Metrik");
46         ImGui::TableSetupColumn("Anlık");
47         ImGui::TableSetupColumn("Tepe");
48         ImGui::TableHeadersRow();
49         const char* names[] = {"CPU", "RAM"};
50         ScrollBuffer* bufs[] = {&cpu, &ram};
51         for (int i = 0; i < 2; i++) {
52             ImGui::TableNextRow();
53             ImGui::TableNextColumn(); ImGui::Text("%s", names[i]);
54             ImGui::TableNextColumn(); ImGui::Text("%.1f%%", bufs[i]->Last());
55             ImGui::TableNextColumn(); ImGui::Text("%.1f%%", bufs[i]->Max());
56         }
57         ImGui::EndTable();
58     }
59     ImGui::Text("FPS: %.0f (%.2f ms)", ImGui::GetIO().Framerate,
60               1000.0f / ImGui::GetIO().Framerate);
61     ImGui::End();
62 });
63 }
64

```

Halka tampon

PlotLines'in values_offset parametresi tam da bir halka tampon için tasarlanmıştır: son yazılan konumu verirsiniz, ImGui diziyi o noktadan itibaren dairesel okur; böylece grafik soldan sağa akar ve en eski örnek düşer. Kaydırmalı grafiklerin en verimli yöntemidir — her kare diziyi kaydırmanıza gerek kalmaz.

24. Proje 4: Renk Paleti Tasarımcısı

Renk seçicilerini, DrawList çizimini ve panoya kopyalamayı birleştiren bir araç. Kullanıcı bir renk seçer, palete ekler, paleti kutucuklar hâlinde görür ve seçilen rengin hex kodunu kopyalar.

24.1 Tam Kaynak

```

1  #include "app.h"
2  #include <vector>
3
4  static ImU32 ToU32(const float c[4]) {
5      return IM_COL32((int)(c[0]*255), (int)(c[1]*255),
6                     (int)(c[2]*255), (int)(c[3]*255));
7  }
8
9  int main() {
10     App app("Palet Tasarimcisi");
11     float current[4] = {0.85f, 0.35f, 0.25f, 1.0f};
12     std::vector<ImVec4> palette;
13
14     app.Run([&]() {
15         ImGui::Begin("Palet");
16
17         // Büyük secici

```

```

18     ImGui::ColorPicker4("#picker", current,
19         ImGuiColorEditFlags_NoSidePreview | ImGuiColorEditFlags_DisplayHex);
20
21     // Hex kodu ve panoya kopyala
22     int r = (int)(current[0]*255), g = (int)(current[1]*255),
23         b = (int)(current[2]*255);
24     char hex[16]; sprintf(hex, sizeof(hex), "%02X%02X%02X", r, g, b);
25     ImGui::Text("Hex: %s", hex);
26     ImGui::SameLine();
27     if (ImGui::SmallButton("Kopyala")) ImGui::SetClipboardText(hex);
28
29     if (ImGui::Button("Palete Ekle")) {
30         palette.push_back(ImVec4(current[0], current[1], current[2], 1));
31     }
32     ImGui::SameLine();
33     if (ImGui::Button("Paleti Temizle")) palette.clear();
34
35     ImGui::Separator();
36     ImGui::Text("Palet (%d renk) -- tiklayınca sec, sag-tik sil:",
37         (int)palette.size());
38
39     // Kutucuklari DrawList ile ciz, satir basina 8 tane
40     ImVec2 start = ImGui::GetCursorScreenPos();
41     ImDrawList* dl = ImGui::GetWindowDrawList();
42     const float box = 36, gap = 6;
43     const int per_row = 8;
44     int remove = -1;
45     for (int i = 0; i < (int)palette.size(); i++) {
46         int col = i % per_row, row = i / per_row;
47         ImVec2 p0(start.x + col*(box+gap), start.y + row*(box+gap));
48         ImVec2 p1(p0.x + box, p0.y + box);
49
50         float c[4] = {palette[i].x, palette[i].y, palette[i].z, 1};
51         dl->AddRectFilled(p0, p1, ToU32(c), 4.0f);
52         dl->AddRect(p0, p1, IM_COL32(255,255,255,60), 4.0f);
53
54         // Tiklanabilir alan
55         ImGui::SetCursorScreenPos(p0);
56         ImGui::PushID(i);
57         ImGui::InvisibleButton("sw", ImVec2(box, box));
58         if (ImGui::IsItemClicked(ImGuiMouseButton_Left)) {
59             current[0]=palette[i].x; current[1]=palette[i].y;
60             current[2]=palette[i].z;
61         }
62         if (ImGui::IsItemClicked(ImGuiMouseButton_Right)) remove = i;
63         ImGui::PopID();
64     }
65     if (remove >= 0) palette.erase(palette.begin() + remove);
66
67     // Kutucuk alani icin yer ac
68     int rows = ((int)palette.size() + per_row - 1) / per_row;
69     ImGui::SetCursorScreenPos(start);
70     ImGui::Dummy(ImVec2(per_row*(box+gap), rows*(box+gap) + 4));
71
72     ImGui::End();
73 });
74 }

```

İpucu

ImGui::SetClipboardText ve GetClipboardText platformdan bağımsız pano erişimi sağlar — backend bunu işletim sistemine bağlar. Araç geliştirirken “hex kodu kopyala”, “JSON’u kopyala” gibi küçük kolaylıklar kullanıcıyı çok mutlu eder.

25. Proje 5: Hex / Bellek Görüntüleyici

Ters mühendislik ve dosya araçlarının klasığı: bir bayt tamponunu onaltılık (hex) ve ASCII olarak yan yana gösteren görüntüleyici. ImGuiListClipper sayesinde megabaytlarca veriyi akıcı gösterir — yalnızca görünen satırlar çizilir.

25.1 Tam Kaynak

```

1  #include "app.h"
2  #include <vector>
3  #include <cstdint>
4
5  int main() {
6      App app("Hex Goruntuleyici");
7
8      // Ornek veri: 256 KB rastgele-benzeri desen
9      std::vector<uint8_t> mem(256 * 1024);
10     for (size_t i = 0; i < mem.size(); i++)
11         mem[i] = (uint8_t)((i * 37 + (i >> 4)) & 0xFF);
12
13     const int cols = 16;    // satir basina bayt
14
15     app.Run([&]() {
16         ImGui::Begin("Bellek");
17         ImGui::Text("Boyut: %zu bayt (%.1f KB)", mem.size(), mem.size()/1024.0f);
18         ImGui::Separator();
19
20         int total_rows = (int)((mem.size() + cols - 1) / cols);
21
22         ImGui::BeginChild("hexscroll", ImVec2(0,0), false,
23             ImGuiWindowFlags_HorizontalScrollbar);
24         ImGui::PushFont(nullptr); // esit genislikli font ideal olurdu
25
26         ImGuiListClipper clipper;
27         clipper.Begin(total_rows);
28         while (clipper.Step()) {
29             for (int row = clipper.DisplayStart; row < clipper.DisplayEnd; row++) {
30                 size_t base = (size_t)row * cols;
31
32                 // Adres
33                 ImGui::Text("%08zX: ", base);
34                 ImGui::SameLine();
35
36                 // Hex sutunlari
37                 char hexpart[64] = "";
38                 for (int c = 0; c < cols; c++) {
39                     char b[4];
40                     if (base + c < mem.size())
41                         snprintf(b, sizeof(b), "%02X ", mem[base + c]);

```

```

42         else
43             snprintf(b, sizeof(b), "    ");
44             strcat(hexpart, b);
45     }
46     ImGui::TextUnformatted(hexpart);
47     ImGui::SameLine();
48
49     // ASCII sutunu
50     char ascii[32] = " |";
51     int len = 2;
52     for (int c = 0; c < cols && base + c < mem.size(); c++) {
53         uint8_t ch = mem[base + c];
54         ascii[len++] = (ch >= 32 && ch < 127) ? (char)ch : '.';
55     }
56     ascii[len++] = '|'; ascii[len] = '\0';
57     ImGui::TextUnformatted(ascii);
58 }
59 }
60 ImGui::PopFont();
61 ImGui::EndChild();
62 ImGui::End();
63 });
64 }

```

Bilgi

Buradaki püf nokta ImGuiListClipper'dır: 16.384 satırın (256 KB / 16) tamamını çizmeye çalışsaydık her kare 16 binden fazla Text çağrısı yapardık. Clipper ile yalnızca ekrandaki ~40 satır çizilir; kaydırma çubuğu yine de tüm veriyi doğru temsil eder. Bu, ImGui'de büyük veri göstermenin altın kuralıdır.

Dikkat

Gerçek bir hex editöründe *eşit genişlikli* (monospace) font şarttır, yoksa sütunlar hizalanmaz. Kısım 3'teki font yükleme yöntemiyle bir monospace TTF (örn. DejaVu Sans Mono) yükleyip PushFont ile bu pencerede kullanın.

26. Proje 6: Basit Çizim Tuvali

DrawList ve fare etkileşimini birleştiren bir vektör çizim uygulaması. Kullanıcı fareyle serbest çizgiler çizer (stroke), rengi ve kalınlığı seçer, son çizgiyi geri alır ve tuvali temizler. Grafik editörlerinin çekirdek mantığını gösterir.

26.1 Tam Kaynak

```

1  #include "app.h"
2  #include <vector>
3
4  struct Stroke {
5      std::vector<ImVec2> points;    // tuval-yerel koordinatlar
6      ImU32 color;
7      float thickness;
8  };
9
10 int main() {

```



```

11 App app("Cizim Tuvali");
12 std::vector<Stroke> strokes;
13 float draw_col[4] = {1.0f, 0.8f, 0.2f, 1.0f};
14 float thickness = 3.0f;
15 bool drawing = false;
16
17 app.Run([&]() {
18     ImGui::Begin("Tuval");
19
20     // Arac cubugu
21     ImGui::ColorEdit3("Renk", draw_col, ImGuiColorEditFlags_NoInputs);
22     ImGui::SameLine(); ImGui::SetNextItemWidth(120);
23     ImGui::SliderFloat("Kalinlik", &thickness, 1.0f, 20.0f);
24     ImGui::SameLine();
25     if (ImGui::Button("Geri Al") && !strokes.empty()) strokes.pop_back();
26     ImGui::SameLine();
27     if (ImGui::Button("Temizle")) strokes.clear();
28
29     // Tuval alanı
30     ImVec2 p0 = ImGui::GetCursorScreenPos();
31     ImVec2 sz = ImGui::GetContentRegionAvail();
32     if (sz.x < 50) sz.x = 50; if (sz.y < 50) sz.y = 50;
33     ImVec2 p1(p0.x + sz.x, p0.y + sz.y);
34
35     ImDrawList* dl = ImGui::GetWindowDrawList();
36     dl->AddRectFilled(p0, p1, IM_COL32(25,25,30,255));
37     dl->AddRect(p0, p1, IM_COL32(120,120,130,255));
38
39     // Etkilesim
40     ImGui::InvisibleButton("tuval", sz,
41         ImGuiButtonFlags_MouseButtonLeft);
42     ImVec2 mouse = ImGui::GetIO().MousePos;
43     ImVec2 local(mouse.x - p0.x, mouse.y - p0.y); // tuval-yerel
44
45     ImU32 col = IM_COL32((int)(draw_col[0]*255), (int)(draw_col[1]*255),
46         (int)(draw_col[2]*255), 255);
47
48     if (ImGui::IsItemActive() && ImGui::IsMouseDown(ImGuiMouseButton_Left)) {
49         if (!drawing) {
50             strokes.push_back({}, col, thickness); // yeni stroke basla
51             drawing = true;
52         }
53         // Ayni noktayi tekrar eklemek icin kucuk esik
54         auto& pts = strokes.back().points;
55         if (pts.empty() ||
56             fabsf(pts.back().x-local.x)+fabsf(pts.back().y-local.y) > 2.0f)
57             pts.push_back(local);
58     } else {
59         drawing = false;
60     }
61
62     // Cizimi kirp ve ciz (tuval disina tasma sin)
63     dl->PushClipRect(p0, p1, true);
64     for (auto& s : strokes) {
65         for (size_t i = 1; i < s.points.size(); i++) {
66             ImVec2 a(p0.x + s.points[i-1].x, p0.y + s.points[i-1].y);
67             ImVec2 b(p0.x + s.points[i].x, p0.y + s.points[i].y);
68             dl->AddLine(a, b, s.color, s.thickness);
69         }

```

```

70     }
71     dl->PopClipRect();
72
73     ImGui::End();
74 });
75 }

```

Yerel vs ekran koordinatı

Noktaları *tuval-yerel* olarak sakladık (tuvalin sol-üstüne göre), çizerken p0'ı ekleyerek ekran uza-yına çevirdik. Böylece pencere taşınırsa çizim onunla birlikte gelir — ekran koordinatı saklasaydık pencere hareket edince çizim yerinde kalırdı. Bu ayrım, tüm tuval/harita/node editörlerinde temel bir tasarım kararıdır.

27. Proje 7: Oyun Hata Ayıklama HUD'u

Son projemiz, ImGui'nin en yaygın kullanım alanını temsil eder: bir oyun/render döngüsünün üz-erine bindirilen hata ayıklama arayüzü. Şeffaf bir üst-katman (overlay), gerçek zamanlı ayar panelleri ve bir varlık (entity) denetleyicisi içerir. Burada “oyun” sahte bir simülasyondur; önemli olan HUD kalıplarıdır.

27.1 Tam Kaynak

```

1  #include "app.h"
2  #include <vector>
3  #include <cmath>
4
5  struct Entity {
6      char name[32];
7      float x, y;
8      float vx, vy;
9      bool alive = true;
10 };
11
12 int main() {
13     App app("Oyun Debug HUD");
14
15     // Sahte oyun durumu
16     std::vector<Entity> entities;
17     for (int i = 0; i < 6; i++) {
18         Entity e; snprintf(e.name, sizeof(e.name), "Dusman_%d", i);
19         e.x = 100 + i*90; e.y = 200;
20         e.vx = cosf(i)*60; e.vy = sinf(i)*60;
21         entities.push_back(e);
22     }
23     int selected = -1;
24     bool paused = false;
25     float sim_speed = 1.0f;
26     bool show_overlay = true, show_inspector = true;
27
28     app.Run([&]() {
29         ImGuiIO& io = ImGui::GetIO();
30         float dt = paused ? 0.0f : io.DeltaTime * sim_speed;
31
32         // --- "Oyun" mantigi ---

```

```

33     for (auto& e : entities) {
34         if (!e.alive) continue;
35         e.x += e.vx * dt; e.y += e.vy * dt;
36         if (e.x < 0 || e.x > 1280) e.vx = -e.vx;
37         if (e.y < 0 || e.y > 720) e.vy = -e.vy;
38     }
39
40     // --- Varliklari dogrudan ekrana ciz (arka-plan DrawList) ---
41     ImDrawList* bg = ImGui::GetBackgroundDrawList();
42     for (int i = 0; i < (int)entities.size(); i++) {
43         Entity& e = entities[i];
44         if (!e.alive) continue;
45         ImU32 col = (i == selected) ? IM_COL32(255,220,0,255)
46                                     : IM_COL32(220,80,80,255);
47         bg->AddCircleFilled(ImVec2(e.x, e.y), 14, col);
48         bg->AddText(ImVec2(e.x-14, e.y+16), IM_COL32(255,255,255,200), e.name);
49     }
50
51     // --- 1) Ust menu ---
52     if (ImGui::BeginMainMenuBar()) {
53         if (ImGui::BeginMenu("Debug")) {
54             ImGui::MenuItem("FPS Overlay", nullptr, &show_overlay);
55             ImGui::MenuItem("Inspector", nullptr, &show_inspector);
56             ImGui::EndMenu();
57         }
58         ImGui::EndMainMenuBar();
59     }
60
61     // --- 2) Seffaf FPS overlay (sag ust kose) ---
62     if (show_overlay) {
63         ImGuiViewport* vp = ImGui::GetMainViewport();
64         ImGui::SetNextWindowPos(
65             ImVec2(vp->WorkPos.x + vp->WorkSize.x - 10, vp->WorkPos.y + 10),
66             ImGuiCond_Always, ImVec2(1, 0)); // sag-ust hizala
67         ImGui::SetNextWindowBgAlpha(0.35f);
68         ImGuiWindowFlags f = ImGuiWindowFlags_NoDecoration
69             | ImGuiWindowFlags_NoMove | ImGuiWindowFlags_NoSavedSettings
70             | ImGuiWindowFlags_NoNav | ImGuiWindowFlags_AlwaysAutoResize;
71         ImGui::Begin("overlay", nullptr, f);
72         ImGui::Text("FPS: %.0f (%.2f ms)", io.Framerate, 1000.0f/io.Framerate);
73         int alive = 0; for (auto& e : entities) if (e.alive) alive++;
74         ImGui::Text("Canli varlik: %d / %d", alive, (int)entities.size());
75         ImGui::Text("Durum: %s", paused ? "DURAKLATI" : "CALISIYOR");
76         ImGui::End();
77     }
78
79     // --- 3) Kontrol paneli ---
80     ImGui::Begin("Simulasyon");
81     if (ImGui::Button(paused ? "Devam" : "Duraklat")) paused = !paused;
82     ImGui::SameLine();
83     if (ImGui::Button("Sifirla")) for (auto& e : entities) e.alive = true;
84     ImGui::SliderFloat("Sim Hizi", &sim_speed, 0.0f, 3.0f);
85     ImGui::End();
86
87     // --- 4) Varlik denetleyicisi (inspector) ---
88     if (show_inspector) {
89         ImGui::Begin("Inspector");
90         if (ImGui::BeginTable("ent", 4, ImGuiTableFlags_Borders
91             | ImGuiTableFlags_RowBg)) {

```

```

92     ImGui::TableSetupColumn("Ad");
93     ImGui::TableSetupColumn("Konum");
94     ImGui::TableSetupColumn("Hiz");
95     ImGui::TableSetupColumn("Islem");
96     ImGui::TableHeadersRow();
97     for (int i = 0; i < (int)entities.size(); i++) {
98         Entity& e = entities[i];
99         ImGui::TableNextRow();
100        ImGui::PushID(i);
101        ImGui::TableNextColumn();
102        if (ImGui::Selectable(e.name, selected == i,
103            ImGuiSelectableFlags_SpanAllColumns))
104            selected = i;
105        ImGui::TableNextColumn(); ImGui::Text("%.0f, %.0f", e.x, e.y);
106        ImGui::TableNextColumn(); ImGui::Text("%.0f, %.0f", e.vx, e.vy);
107        ImGui::TableNextColumn();
108        if (ImGui::SmallButton(e.alive ? "Oldur" : "Diri"))
109            e.alive = !e.alive;
110        ImGui::PopID();
111    }
112    ImGui::EndTable();
113 }
114 // Secili varligin detayli duzenlemesi
115 if (selected >= 0 && selected < (int)entities.size()) {
116     ImGui::SeparatorText("Secili Varlik");
117     Entity& e = entities[selected];
118     ImGui::InputText("Ad", e.name, sizeof(e.name));
119     ImGui::DragFloat2("Konum", &e.x, 1.0f);
120     ImGui::DragFloat2("Hiz", &e.vx, 1.0f);
121 }
122 ImGui::End();
123 }
124 });
125 }

```

Bilgi

Bu proje, ImGui'nin oyun geliştirmedeki “süper gücünü” özetler: `GetBackgroundDrawList()` ile oyun dünyasının *üzerine* çizip (varlık etiketleri, çarpışma kutuları, yol bulma ızgaraları), aynı anda yüzen panellerle canlı ayar yapabilirsiniz. `selected` indeksiyle tablodaki seçim ile dünyadaki vurgulama otomatik senkron kalır — çünkü ikisi de *aynı* veriyi her kare yeniden okur. Retained mode'da bu senkronu elle sağlamak zorunda kalırdınız.

KISIM 5

Başvuru ve Kapanış

28. Hızlı Başvuru (Cheat Sheet)

28.1 Sık Kullanılan Çağrılar

Çağrı	İş
Begin/End	Pencere aç/kapat (End daima çağrılır)
Text / TextColored	Metin göster
Button	Düğme; tıklandıysa true
Checkbox / RadioButton	İki/çok durumlu seçim
SliderFloat / DragFloat	Sayısal değer ayarla
InputText	Metin girişi
Combo / ListBox	Açılır/liste seçimi
ColorEdit4	Renk seç
SameLine	Sonraki widget yan yana
Separator / Spacing	Ayraç / boşluk
PushID / PopID	Benzersiz ID kapsamı
BeginTable / EndTable	Tablo
BeginChild / EndChild	Kaydırılabilir alt bölge
IsItemHovered / Clicked	Son öğe durumu
GetWindowDrawList	Özel çizim (DrawList)

28.2 Eşleşmesi Zorunlu Çiftler

Kural	Açıklama
Begin↔End	Pencere: End her zaman (Begin false olsa da).
BeginChild↔EndChild	Child için de End her zaman.
TreeNode→TreePop	Yalnızca TreeNode true dönerse.
BeginTable→EndTable	Yalnızca Begin true dönerse.
Push*↔Pop*	Push sayısı = Pop sayısı (ID, StyleColor, StyleVar).

29. Sık Yapılan Hatalar ve Çözümleri

Belirti	Neden / Çözüm
Döngüdeki düğmeler yanlış çalışıyor	ID çakışması. PushID(i)/PopID() kullan.
Değer bir kare sonra sığıyor	Değeri static yaptın ama iki yerde çizdin; tek kaynağa bağla.
Assert: “mismatched Begin/End”	End'i atladın ya da yanlış eşledin. Çiftleri kontrol et.
Türkçe harfler kutu/boş çıkıyor	Font glyph aralığına Latin Extended-A ekle, UTF-8 kaydet.
Pencere taşınmıyor	Konumu ImGuiCond_Always ile zorluyorsun; _FirstUseEver kullan.
Metin yazarken oyun hareket ediyor	io.WantCaptureKeyboard kontrol edilmedi.
Uzun liste FPS düşürüyor	ImGuiListClipper kullan.
Stil tüm UI'da bozuldu	PushStyleColor/Var sayısı Pop ile eşleşmiyor.
Ekranda hiçbir şey yok	NewFrame/Render sırası veya RenderDrawData eksik.

30. Sonraki Adımlar

Bu rehber Dear ImGui'nin çekirdeğini ve en yaygın kullanım kalıplarını kapsadı. Buradan sonra derinleşmek için:

Konu / Araç	Ne için
<code>imgui_demo.cpp</code>	En iyi öğretmen: her widget'ın canlı örneği ve kaynağı.
ImPlot	Ciddi bilimsel/gerçek zamanlı grafikler.
ImGuizmo	3B sahnede taşı/döndür/ölçekle gizmaları.
imgui-node-editor	Düğüm tabanlı editörler (shader graph vb.).
Docking branch	IDE benzeri çoklu-pencere düzenleri.
Custom backend	Kendi motorunuza (DX12/Vulkan/Metal) entegre etme.
<code>imgui_stdlib.h</code>	<code>std::string</code> ile <code>InputText</code> .

Bilgi

ImGui öğrenmenin en hızlı yolu **yazmaktır**. Bir yan projenizi açın, üstüne bir `ImGui::Begin("Debug")` penceresi koyun ve o projeye özgü değerleri (pozisyonlar, sayaçlar, bayraklar) canlı gösterip ayarlamaya başlayın. Bir kez bu alışkanlığı edindiğinizde, ImGui'siz geliştirme yapmak zor gelecek. İyi çalışmalar!